İNTERNET TEKNOLOJİLERİNE GİRİŞ DERS NOTLARI

Prof. Dr.

Hidayet Takcı

htakci@cumhuriyet.edu.tr

Sivas, 2026

İçindekiler

1. İNTERNET TEKNOLOJİLERİ GİRİŞ 7

1.1. Bilgisayar ağları 7

1.2. Ağ türleri 7

1.3. İnternet 8

1.3.1. Fiziksel altyapı 9

1.3.2. Mantıksal altyapı 10

1.4. Web (World Wide Web – WWW) 12

1.4.1. Web'in İnternet Üzerinde Çalışma Mantığı 13

1.4.2. Web sayfası 13

1.4.3. Web sitesi 14

1.4.4. Web browser 15

1.4.5. URL (Uniform Resource Locator) 15

2. WEB TASARIMI 16

2.1. Web tasarımı nedir? 16

2.1.1. Web tasarımının temelleri 16

2.2. İyi bir tasarımın şartları 16

2.3. Tasarım için genel yöntemler 16

2.3.1. Geçici çözüm (ad-hoc) 16

2.3.2. Metodolojik çözüm 17

2.4. Ön tasarım çalışması 17

2.5. İçerik planlama 18

2.6. Web tasarımında öne çıkan konular 18

2.6.1. Görsel Tasarım (Visual Design) 18

2.6.2. Kullanılabilirlik (Usability) 19

2.6.3. Kullanıcı Deneyimi (UX) 20

2.6.4. Performans & Teknik Kalite 20

2.6.5. Responsive & Erişilebilirlik 21

2.6.6. İçerik & SEO Uyum 22

2.6.7. Genel tasarım düzeltmeleri 22

3. HTML5 24

3.1. Giriş 24

3.2. HTML doküman yapısı 24

3.3. HTML'de açıklama/yorum satırları 25

3.4. Başlık etiketleri (headings) 25

3.5. Metin etiketleri ... 26
3.5.1. Paragraf etiketi ... 26
3.5.2. Önceden tanımlı paragraf etiketi ... 26
3.5.3. Satır sonu etiketi ... 26
3.5.4. Yatay çizgi etiketi ... 27
3.5.5. Vurgulama (Highlighting) etiketi ... 27
3.5.6. Kalın, italik, altı çizgi etiketi ... 27
3.5.7. Kısaltma (abbrevation) etiketi ... 27
3.5.8. Ekleme, silme, üstünü çizme ... 27
3.5.9. Üst ve alt işaretleri (Superscript and Subscript): ... 28
3.6. Bağlantılar (links) ... 28
3.7. Dosyayı bağlantı ile dâhil etme ... 30
3.8. Listeler ... 30
3.9. Tablolar ... 31
3.9.1. Colspan özelliği ... 32
3.9.2. Rowspan özelliği ... 33
3.10. Resimler ... 33
3.11. Formlar ... 34
3.11.1. Form elemanları ... 35
3.11.2. Giriş Doğrulama ... 41
3.11.3. Form gönderme ... 42
3.11.4. Form elemanlarının gruplanması ... 42
3.12. Div etiketi ... 43
3.13. HTML ve CSS ... 43
3.14. HTML ve Javascript ... 44
3.15. Semantik elemanlar ... 45
3.15.1. <nav> element ... 45
3.15.2. <article> element ... 46
3.15.3. <section> element ... 46
3.15.4. <aside> element ... 47
3.15.5. <main> element ... 47
3.15.6. <header> element ... 48
3.15.7. <footer> element ... 48
3.15.8. <figure> element ... 48
3.15.9. <figcaption> element ... 49

3.15.10. <summary> element ... 49
3.15.11. <label> element ... 50
3.15.12. <output> element ... 50
3.16. Ses kontrolleri ... 51
3.17. Video kontrolleri ... 51
3.18. Seçim (select) kontrolleri ... 52
3.19. Datalist element ... 54
3.20. Embed element ... 55
3.21. IFrames kullanımı ... 55
4. CSS ... 58
4.1. CSS Nedir? ... 58
4.2. CSS yapısı ... 58
4.3. Bir elemana (element) birden çok stil özelliği uygulama ... 58
4.4. Birden çok elaman için stil uygulama ... 59
4.5. Stil ekleme yöntemleri ... 59
4.5.1. Satır içi stil ekleme ... 59
4.5.2. Dâhili stil ekleme ... 60
4.5.3. Harici stil ekleme ... 60
4.6. CSS Seçicileri ... 61
4.6.1. Class seçiciler ... 62
4.6.2. ID seçiciler ... 62
4.7. Metinler ... 63
4.7.1. Metin biçimlendirme stilleri ... 63
4.7.2. Metin boyutlandırma stilleri ... 65
4.8. Renkler ... 68
4.8.1. Metin renklendirmede genel kullanım: ... 68
4.8.2. Geri plana renk verme ... 68
4.9. Resimler ... 68
4.9.1. Temel özellikler: ... 69
4.9.2. Yuvarlak köşeler ... 69
4.9.3. Gölge efekti ... 69
4.9.4. Resim ortalama ... 70
4.9.5. Resim arkaplanı ... 70
4.10. Listeler ... 70
4.10.1. Varsayılan işaretleri kaldırma ... 70

4.10.2. Liste elemanları başına ikon yerleştirme ... 71
4.10.3. Yatay menü listesi ... 71
4.10.4. Liste elemanlarına hover efekti ... 71
4.10.5. Numaralı liste ... 72
4.11. Tablolar ... 72
4.11.1. Kenarlık birleştirme ... 72
4.11.2. Çerçeve özellikleri ... 72
4.11.3. Başlık satırı renklendirme ... 73
4.11.4. Çizgili tablo ... 73
4.11.5. Satır için hover ... 73
4.11.6. Responsive tablo ... 73
4.11.7. Toplu örnek ... 74
4.12. CSS kutu (box) modeli ... 75
4.12.1. Padding (dolgu) ... 76
4.12.2. Border (kenar) ... 78
4.12.3. Margin ... 81
4.12.4. Dimemsions ... 83
4.13. Floating elements (yüzen elemanlar) ... 84
4.14. Positioning elements (konumlanan elemanlar) ... 86
4.14.1. Position: static ... 86
4.14.2. Position: relative ... 87
4.14.3. Position: absolute ... 87
4.14.4. Position: fixed ... 87
4.14.5. Position: sticky ... 88
4.15. CSS Flexbox ... 88
4.16. Grid ... 91
5. JAVASCRIPT ... 95
5.1. Javascript Giriş ... 95
5.1.1. Javascript değişkenleri ... 96
5.1.2. Veri tipleri ... 97
5.1.3. Operatörler ve yapılar ... 98
5.1.4. Fonksiyonlar ... 98
5.1.5. Koşullar ve döngüler ... 99
5.1.6. Nesneler ... 100
5.2. Document Object Model (DOM) ... 101

5.2.1. DOM nesneleri 101
5.2.2. HTML DOM: Document 102
5.3. Javascript olayları (events) 103
5.3.1. Click olayı ile ilgili örnekler 103
5.3.2. Mouse olayı ile ilgili örnekler 105
5.3.3. Klavye ile ilgili olay örnekleri 106
5.3.4. Hesap makinesi örneği 107
5.4. LocalStorage kullanımı 107
5.5. Fetch API ile veri çekme 107
5.6. Hata yakalama 109
5.7. Modüle kullanımı 109

1. İNTERNET TEKNOLOJİLERİ GİRİŞ

1.1. Bilgisayar ağları

İletişim (e-mail, canlı sohbet), paylaşım (yazıcı, dosya, bellek veya işlemci) ve takas (bilgi/belge gönder /al) yapmak amacıyla, iki veya daha fazla bilgisayarın bir araya gelmesiyle oluşan yapılardır.

1.2. Ağ türleri

Ağlar farklı ölçeklerde olabilir. Bunların öne çıkan tipleri aşağıda verilmiştir:

Küçük ev ağları (**Small Home**) – birkaç bilgisayarı birbirine ve Internet'e bağlayan ağlardır.

Küçük ofis / ev ofisi (**SOHO**) – bir ev veya uzak ofisteki bilgisayarların bir şirket ağına bağlanmasını sağlayan ağlardır. Uzaktan çalışanlar için en uygun ağ türüdür.

Orta / Büyük ölçekli ağlar (**LAN / MAN**) – bir yerel alan ağındaki birden çok bilgisayarın birbirine bağlanmasıyla ortaya çıkan ağlara LAN, daha uzak bölgelerdeki ağların birbiriyle bağlanması sonucu ortaya çıkan ağlara da MAN ağı adı verilir.

Geniş alan ağları – dünya çapında yüz milyonlarca bilgisayarı birbirine bağlayan ağlardır.

1.3. İnternet

Dünya çapında milyarlarca bilgisayar, sunucu, telefon ve diğer akıllı cihazı birbirine bağlayan küresel bir iletişim ağıdır. Bu ağ sayesinde insanlar ve sistemler arasında **veri, bilgi ve hizmet paylaşımı** yapılır**.**

İnternet, 1969'da ARPA (Amerikan Savunma Bakanlığı) tarafından geliştirilen ARPANET isimli yapıdan günümüze evrilmiştir. İnternetin iki temel alt yapısı vardır:

- Fiziksel alt yapı (fiber optik kablolar, uydular, modemler, yönlendiriciler)
- Mantıksal alt yapı (TCP/IP, IP adresleri, yönlendirme protokolleri)

1.3.1. Fiziksel altyapı

İnternetin fiziksel altyapısında; **kablolar, kablosuz iletim ortamları, uydular ve ağ cihazları** bulunur. Fiziksel altyapı iki uç arasında iletim görevini yerine getirir.

- **Fiber optik kablolar**
  - İnternet omurgasının temelini oluşturur.
  - Kıtalar arası denizaltı kablolar bu türdendir.
  - Işık sinyalleriyle çok yüksek hızda veri iletir.

- **Bakır kablolar**
  - DSL, Ethernet gibi teknolojilerde kullanılır.
  - Fiberden daha düşük hız ve kapasiteye sahiptir.

- **Kablosuz iletim ortamları**
  - Wi-Fi, hücresel ağlar (4G/5G), mikrodalga
  - Radyo dalgalarıyla veri iletimi (RFID)

- **Uydular**
  - Uzak ve erişimi zor bölgeler için kullanılır (Starlink uyduları gibi)
  - Gecikme (latency) yüksektir

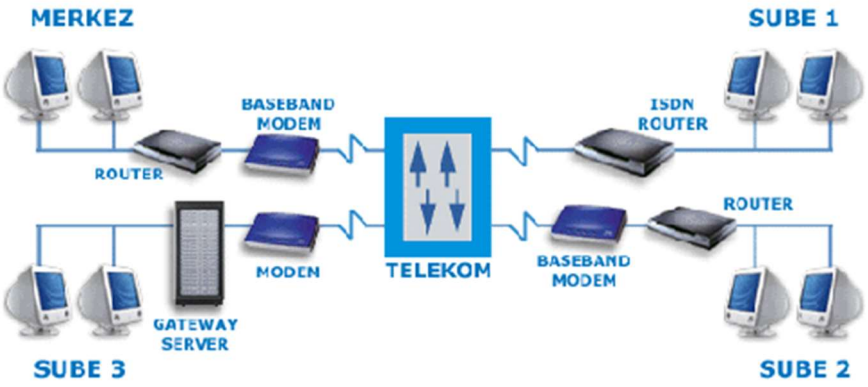
- **Ağ cihazları**
  - Hub, anahtar (switch), repeater, bridge, modem, yönlendirici (router)
  - Veriyi doğru hedefe yönlendirir

1.3.1.1. İnternete bağlantı yöntemleri

İnterneti kullananların ihtiyaçlarına göre bağlantı yöntemleri değişiklik gösterir.

Kiralık hat (leased line)

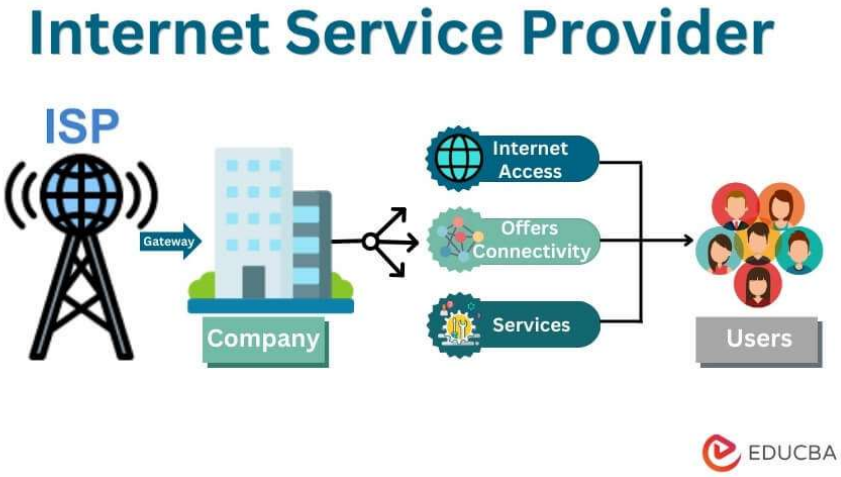
Ticari bir sözleşmeye göre sağlanan iki veya daha fazla kurum/konum arasındaki özel bir haberleşme devresidir. Bazen veri hattı olarak da isimlendirilir. Genellikle, kiralık hatlar işletmeler tarafından coğrafi olarak uzak ofisleri birbirine bağlamak için kullanılır.



Güvenliğin önem kazandığı askeri birimler de kendi arasındaki haberleşmesini kiralık hatlar üzerinden ve herkese açık olmayan ağ üzerinde yürütürler.

İnternet Servis Sağlayıcı (İSS veya ISP)

Şirketlere ve kişilere internet bağlantısı sağlayan kurumlardır. İnternet servis sağlayıcı firmalar internet erişimi hizmeti ve veri barındırma hizmetleri verirler.



İnternete bağlanmak isteyen herkes kiralık hat kullanacak kadar zengin olmayabilir. Ayrıca öyle bir ihtiyaçları da olmayabilir. Öylesi zamanlarda daha makul ve daha az güvenli çözümler kullanabilirler. İnternet servis sağlayıcıları bu noktada devreye girer.

1.3.2. Mantıksal altyapı

İnternetin mantıksal altyapısı; **adresleme, paketleme, iletim ve yönlendirme işlemlerini yöneten kurallardan** oluşur**. Bu kurallar** İnternet protokolleri olarak bilinir.

IP (Internet Protocol)

Verilerin hedefe ulaşabilmesi için kaynak ile hedef arasında bağlantı kurulmasını yöneten kurallar sağlar.

- Verilerin hedefteki bir cihaza ulaşabilmesi için öncelikle her cihazın benzersiz bir adres almasını IP protokolü sağlar.
- IPv4 ve IPv6 gibi iki farklı sürümü vardır.

TCP / UDP

Bağlantı kurulduktan sonra iletimi yöneten kurallar sağlar. TCP ve UDP gibi iki yönteme göre çalışır.

- **TCP**: Güvenilir, sıralı ve hatasız iletim sağlar. Gönderilen verinin doğru şekilde gönderilip gönderilmediği kontrol edilir.
- **UDP**: Hızlı ama bağlantısız iletim. Veri gönderilir ama gönderimin doğru olup olmadığı kontrol edilmez.

Paketleme (Packet Switching)

İletim esnasında verinin nasıl organize edileceği ve hangi yolları takip edeceği ile ilgilidir.

- Veri küçük paketlere bölünerek iletilir
- Paketler farklı yollardan gidip hedefte birleştirilir

Yönlendirme (Routing)

Kaynak ile hedef arasındaki en optimum yolun seçilmesinde kullanılır.

- Router'lar paketler için en uygun yolu seçer
- BGP, OSPF gibi yönlendirme protokolleri kullanılır

DNS (Domain Name System)

IP adresleri ile alan adları arasında dönüşüm sağlar. IP adresleri akılda kalıcı olmadığı için IP adresleri yerine takma isimler üretir.

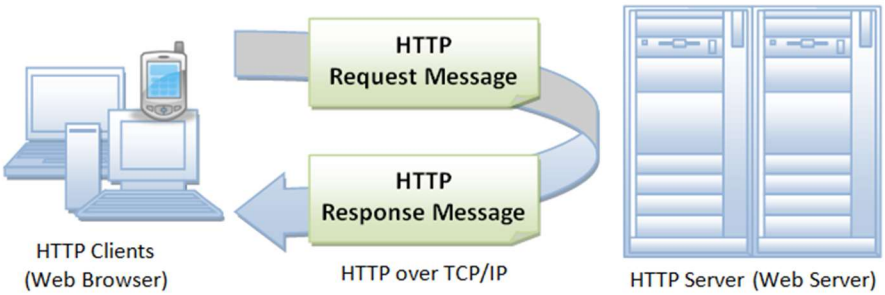
- Alan adlarını IP adreslerine çevirir
- Örn: google.com → 142.250.x.x

Portlar ve Oturum Yönetimi

- Aynı IP üzerinde birden fazla hizmetin çalışmasını sağlar
- Örn: HTTP (80), HTTPS (443)

HTTP / HTTPS

- Web sayfalarının iletimine dair kurallar içerir
- HTTPS, şifreli iletişim sağlar



FTP / SFTP

- Dosya aktarımının kurallarını sağlar.
- SFTP, güvenli dosya aktarımı yapar.

SMTP

- E-posta gönderimine dair kuralları içerir.

POP3 / IMAP

- E-posta alma ve yönetme kurallarını kapsar.

ARP

- IP adresini MAC adresine, MAC adresini IP adresine çevirir.

ICMP

- Hata ve durum mesajları (ping) sağlar.

1.4. Web (World Wide Web – WWW)

İnternet, dünya genelinde bilgisayar ağlarını birbirine bağlayan **küresel iletişim altyapısıdır**. Web (World Wide Web) ise bu altyapı üzerinde çalışan ve kullanıcıların bilgiye **tarayıcılar aracılığıyla erişmesini sağlayan bir sistemdir**.

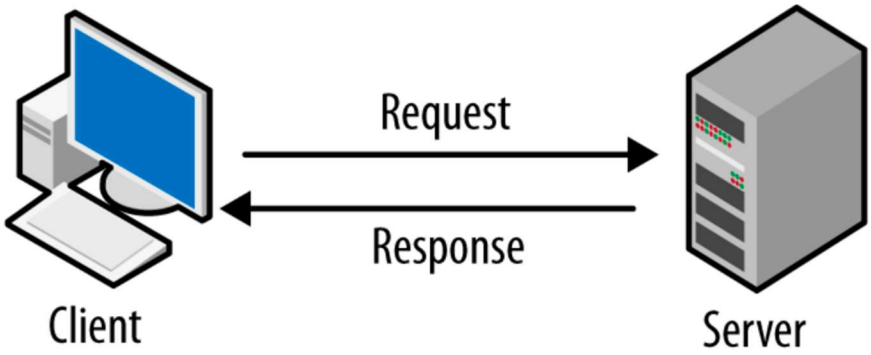
Başka bir ifadeyle, **internet web'in çalıştığı ortam**, web ise **internetin sunduğu hizmetlerden biridir**. İnternet olmadan web çalışamaz; ancak internet, web dışındaki pek çok hizmeti de barındırır (e-posta, dosya transferi, mesajlaşma vb.).

1.4.1. Web'in İnternet Üzerinde Çalışma Mantığı

Web, internetin sunduğu **IP, TCP ve DNS** gibi temel protokolleri kullanarak çalışır. Bu protokoller sayesinde:

- Web sunucuları adreslenir,
- İstemci ile sunucu arasında bağlantı kurulur,
- Web içerikleri güvenli ve düzenli biçimde iletilir.

Web, **istemci–sunucu modeli** temel alınarak tasarlanmıştır. İstemci-sunucu modeli bir tarafın talep gönderen diğer tarafın da talep karşılayan rolünde olduğu temel iletişim modelidir. Talebin türü veri, bilgi ya da hizmet olabilir.



İstemci (Client)

Sunucuya istek gönderen ve gelen cevabı alan/görüntüleyen ağ elemanıdır.

- Genellikle **web tarayıcısıdır** (Chrome, Firefox vb.)
- Kullanıcıdan gelen isteği sunucuya iletir
- Sunucudan gelen yanıtı kullanıcıya gösterir

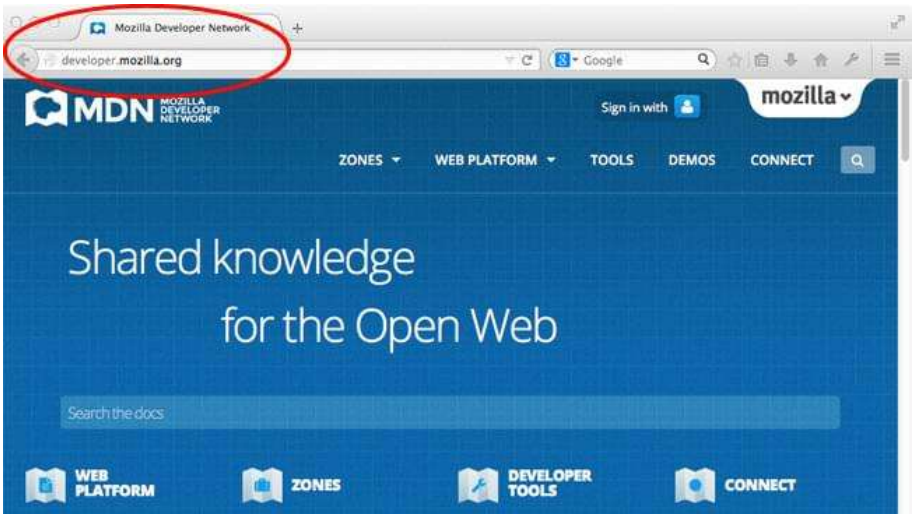
Sunucu (Server)

İstemciden gelen veri ya da işlem taleplerini karşılayan yazılım ya da donanımlardır.

- Web sayfalarını ve kaynakları barındırır
- İstemciden gelen HTTP/HTTPS isteklerini işler
- HTML, CSS, görsel veya veri yanıtı üretir

1.4.2. Web sayfası

Bir web sayfası bir web göz atıcı (browser) aracılığıyla ulaşılabilen, world wide web ile uyumlu yarı yapısal ya da yapısal olmayan bir dokümandır.

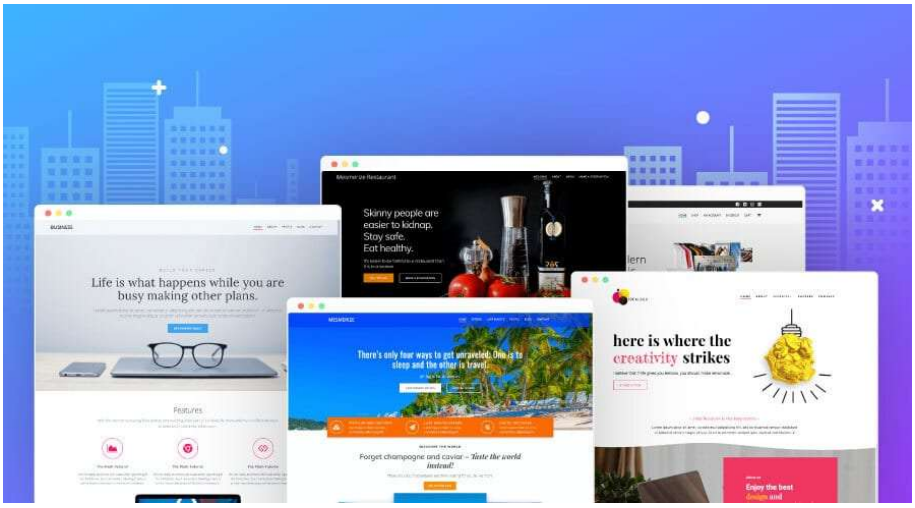


1.4.2.1. HTML

Hiper Metin İşaretleme Dili (HTML, HyperText Markup Language) web sayfalarını oluşturmak için kullanılan standart metin işaretleme dilidir. Dilin son sürümü HTML5'tir.

1.4.3. Web sitesi

Aynı web sunucu üzerinde yer alan web sayfalarının, resimlerin, stillerin, script dosyalarının ve diğer dosyaların bir koleksiyonudur.



Web siteleri web sunucuları tarafından barındırılır.

1.4.4. Web browser

Web sayfalarını görmek için kullanılan özel yazılımlardır. HMTL etiketlerini yorumlamakta kullanılır. Google Chrome, Internet Explorer, Microsoft Edge, Mozilla Firefox vb. türleri vardır.

1.4.5. URL (Uniform Resource Locator)

İnternet üzerindeki bir kaynağı belirtmek için kullanılan tekil adrestir. URL bir alan adı olabileceği gibi alan adı içerisindeki bir dosya yolunu da barındırabilir.

2. WEB TASARIMI

2.1. Web tasarımı nedir?

Kullanıcılara bilgi verme onları ikna etme ya da onları eğitme amacıyla metinlerin, görsellerin ve diğer içeriğin **kullanılabilirlik** ve **erişilebilirlik** ilkelerine uygun olarak web sayfaları üzerinde sunumuna imkân sağlayan çalışma alanıdır.

2.1.1. Web tasarımının temelleri

Web tasarımın amacı: web sitesinde yer alan içeriğin planlanması ve düzenlenmesidir? Aşağıdaki iki sorunun cevabı çalışma boyunca aranır:

- Neler sunulacak? (içerik) (metin, resim, ses, video vs.)
- Nasıl sunulacak? (görsel tasarım, kullanılabilirlik ve erişilebilirlik)

2.2. İyi bir tasarımın şartları

İyi bir tasarım; anlaşılır, ilgi çekici ve kolay kullanılabilir olmalıdır.

Tasarım kuralları faydalıdır fakat yeterli değildir. Çünkü «her duruma uyan tek bir model yoktur».

Yapılacak tasarım içerik üzerinde kontrolü mümkün hale getirmelidir. Kullanıcı sayfa üzerinde kaybolmamalıdır.

2.3. Tasarım için genel yöntemler

2.3.1. Geçici çözüm (ad-hoc)

- Aceleyle bir araya getirilmiş,
- Anında oluşturulmuş,
- “BUGÜN bir web sitesine ihtiyacımız var” talebiyle başlanıp çalakalem ortaya konulmuş sayfalar sağlar.

Dezavantajları

- Birçok yerde "yapım aşamasında – under construction" uyarısı görürsünüz.
- Güncel olmayan içerikle karşılaşırsınız.
- Bol hatalı tasarım (bozuk bağlantılar, bozuk komut dosyaları) bulunur.
- Kafa karıştırıcı bir site kullanıcı karşısına çıkar
- CSS'de yalnızca orijinal tasarımcının anlayabileceği «spagetti kodu» bulunur.
- Güncellenmesi ve bakımı zordur.

Avantajları

Yeterince iyi olmamasına rağmen bazen geçici çözüm ile oluşturulmuş sayfalar daha iyi bir çözüm olabilir (zaman ve maliyet açısından). Aşağıdaki durumlarda geçici çözüm faydalı olabilir:

- Kısa süreli web siteleri
- Çok küçük web siteleri
- Çok özel bir amaç için tasarlanmış siteler (tek bir anket, tek bir sınıf, belirli bir etkinlik vb.)

2.3.2. Metodolojik çözüm

- Planlama yapılmış
- Kalite-güvence sistematiğine uygun sayfalar sağlar.

Hazırlık süreci daha fazla zaman alsa da, iyi düşünülmüş (metodolojik) bir web sitesi:

- Daha az probleme sahiptir
- Daha etkilidir
- Daha anlaşılabilirdir
- Daha kolay yönetilebilir

Bakım maliyetleri düşünüldüğünde metodolojik çözüm her zaman toplamda daha az zaman alacaktır.

2.4. Ön tasarım çalışması

Tasarım çalışmalarına başlamadan önce bazı işler yapılmış olmalıdır.

- Kuruluşunuzun misyonunu düşünün. (**Misyonumuz**, yenilikçi ve güvenilir çözümler üreterek teknoloji ile insan yaşamı arasında sürdürülebilir bir değer köprüsü kurmaktır.)
- Hedef kitleyi tanımlayın (**Hedef Kitlemiz**, teknolojiye ilgi duyan; yapay zekâ, yazılım ve veri analitiği alanlarında kendini geliştirmek isteyen bireyler ile dijital dönüşüm sürecinde yenilikçi çözümler arayan kurum ve kuruluşlardır.)
- Web sitesi için hedefler belirleyin
    - Orta dönemli (**Orta Dönemli Hedefimiz**, dijital platformumuzu kapsamlı bir bilgi ve hizmet merkezine dönüştürerek kullanıcı etkileşimini artırmak,

sürdürülebilir içerik üretimiyle marka bilinirliğini güçlendirmek ve sektörde güvenilir bir referans noktası olmaktır.)

  - Uzun dönemli (**Uzun Dönemli Hedefimiz**, ulusal ve uluslararası düzeyde tanınan, alanında öncü bir dijital platform haline gelerek bilgi üretimi, eğitim, Ar-Ge ve yenilikçi teknolojilerin paylaşımında sürdürülebilir bir ekosistem oluşturmaktır.)

- İçeriği toplayın
  - İçerik hiyerarşisini düzenleyin ve oluşturun
  - İçeriği mantıksal bilgi birimlerine "parçalayın"

2.5. İçerik planlama

(1) Öncelikle kâğıt veya bilgisayar ortamında web sitesi prototipi oluşturun.
Ziyaretçilerinizin bir sayfadan diğerine nasıl ulaşacağına dair bir akış şeması oluşturun.
(2) Web sitesinde ihtiyaç duyduğunuz HTML, PDF, resim, ses ve diğer dosyalar hakkında düşünün.

Nerede konumlanacaklar?

Ziyaretçiler onlara nasıl ulaşacaklar?

(3) Geliştirme ekibinin web sayfalarının hiyerarşisini anlayabilmesi için dosyaları mantıksal olarak düzenleyin.

2.6. Web tasarımında öne çıkan konular

2.6.1. Görsel Tasarım (Visual Design)

Kullanıcıda ilk izlenimi **oluşturan** katmandır. Aşağıdaki kriterler önemlidir.

- Renk uyumu (contrast (açık-koyu farkı, büyük-küçük farkı, font farkı), marka uyumu)
- Tipografi (font seçimi, okunabilirlik)
- Boşluk kullanımı (white space) (margin, padding, satır aralıkları, karmaşa oluşturmamak için boşluk)
- Görsel hiyerarşi (önce görülmesi gerekenin diğerlerinden öne çıkarılması)
- Tutarlılık / uyum (buton, ikon, başlık)

Ölçüm Soruları

- Göz yoruyor mu?
- Ana mesaj ilk 5 saniyede anlaşılabiliyor mu?

Bir web sitesinin içeriği onun nasıl görünmesi gerektiğine etki eder.

- Öğrenciler için tasarlanmış siteler, personel için tasarlanmış sitelerden ve potansiyel öğretim üyeleri için tasarlanmış sitelerden farklı görünmelidir.
- Mevcut çalışanlar için tasarlanan siteler, potansiyel müşteriler için tasarlanan sitelerden farklı görünmelidir.
- İnsanların ürün satın almasını sağlamak için tasarlanmış siteler, bilgi sağlamak için tasarlanmış sitelerden farklı görünmelidir.

Destek ekibi

- Konu uzmanlarından bu konuda yardım alınabilir.
- Sitenin, desteklediği grubun kilit üyeleri (CEO'lar, Bölüm Başkanları, Şefler vb.) tarafından PERİYODİK OLARAK gözden geçirilmesini sağlayın.
- Farklı kişilerin içeriği netlik açısından incelemesini sağlayın.
- Başkalarının gramer için redaksiyon okumasını sağlayın: Taze gözler genellikle sizin görmediklerinizi görür!

2.6.2. Kullanılabilirlik (Usability)

Kullanıcı hedefe ne kadar kolay ulaşıyor?

- Menü yapısı kullanışlı olmalıdır.
- Tıklanabilir alanların netliği (**Tıklanabilir alanların netliği**, bir web ya da mobil arayüzde **kullanıcının hangi öğelere tıklayabileceğini hiçbir tereddüt yaşamadan anlayabilmesidir**.)
- Formlar sade olmalıdır.
- Hata mesajları anlaşılır olmalıdır.
- Öğrenilebilirlik (**Öğrenilebilirlik (Learnability)**, bir web sitesi, yazılım ya da arayüzün **ilk kez kullanan bir kişinin, herhangi bir eğitim veya yardım almadan temel işlemleri ne kadar hızlı ve kolay öğrenebildiğini** ifade eden **kullanılabilirlik (usability) ölçütlerinden biridir**.)

Ölçüm Soruları

- Kullanıcı yardım almadan işlem yapabiliyor mu?
- 3 tıklama kuralı sağlanıyor mu? (Bir kullanıcı, bir web sitesinde aradığı bilgiye veya yapmak istediği işleme **en fazla 3 tıklama** ile ulaşabilmelidir.)

Tavsiyeler:

- Belirli bir tarayıcı için site tasarımı yapılmamalıdır. Her tarayıcı tarafından çalışabilir siteler tasarlanmalıdır.
- Belirli bir insan tipi için site tasarımı yapılmamalıdır. Sitenize erişen kitleyi hayal ederek tasarım yapılmalıdır.
    - Çoğu kullanıcı verileri görsel olarak alır.
    - Çoğu kullanıcı, sitenizin nasıl çalıştığıyla ilgili şeyleri hatırlamak için çaba harcamak istemez.
- Ziyaretçiler sitede kaybolmadan gezinebilmelidir. (menü ve navigasyon desteği)
- Bir hedefe ulaşmak için farklı yöntemler desteklenmelidir. (bir hedef erişmek için farklı seçenekler sunun)
- Ziyaret edilen ve ziyaret edilmeyen bağlantılar, kullanıcıların nerede olduklarını hatırlamalarını kolaylaştırmak için farklı renklerde olmalıdır. (bağlantıların renklendirilmesi)
- Bir grup seçenekteki öğelerin sayısı sınırlandırılmalıdır. (sayfayı mantıksal bölümlere ayırın)

2.6.3. Kullanıcı Deneyimi (UX)

Kullanıcı siteden memnun ayrılıyor mu?

- Akış (user flow)
- Geri bildirimler (hover, loading, success)
- Mikro etkileşimler
- Duygusal etki (trust, profesyonellik)

Ölçüm Soruları

- Kullanıcı yarıda bırakıyor mu?
- Site güven veriyor mu?

2.6.4. Performans & Teknik Kalite

Site ne kadar hızlı ve sorunsuz?

- Sayfa açılış süresi (Kullanıcılar:1 saniye sorunsuz bekler, 10 saniye sıkılmaya başlar, 10 saniyeden fazla terk eder)
- Lighthouse skorları (**Lighthouse skoru**, Google tarafından geliştirilen **Lighthouse** aracının bir web sitesini **performans, erişilebilirlik, SEO ve teknik kalite** açısından otomatik olarak analiz edip **0–100 arası puanlarla** değerlendirmesidir.)

- Kod kalitesi (**Kod kalitesi**, bir yazılımın (web, mobil ya da backend) yalnızca **çalışıyor olmasını değil**, aynı zamanda **okunabilir, sürdürülebilir, güvenli, performanslı ve geliştirilebilir** olmasını ifade eder.)
- Lazy loading (**Lazy loading (tembel yükleme)**, bir web sayfasındaki içeriklerin (özellikle **görseller, videolar, iframe'ler ve ağır bileşenler**) **ilk açılışta değil, ihtiyaç duyulduğunda** yüklenmesi tekniğidir)
- Cache kullanımı (**Cache kullanımı (önbellekleme)**, bir web sitesinin **daha önce üretilmiş veya indirilmiş verileri tekrar tekrar sunucuya gitmeden** kullanmasını sağlayarak **hız, performans ve ölçeklenebilirliği** artıran temel bir tekniktir.)

Ölçüm Soruları

- İlk içerik ne kadar sürede geliyor?
- Mobil ağda çalışıyor mu?

2.6.5. Responsive & Erişilebilirlik

Erişilebilirlik tasarımın "herkes ve her cihaz için uygun olup olmadığı" konusuyla ilgilenir. Erişilebilirlik sorunları kullanımı etkiler. Aşağıda bazı sorunlar verilmiştir:

- Kullanıcıların zayıf görme yeteneği
- Ekran olarak daha küçük, daha eski bilgisayar
- Kullanıcıların zayıf renk algısı
- "Kokteyl partisi" etkisi -- tek bir web sayfasında çok sayıda bilgi

Çözüm için aşağıdakiler yapılmalıdır:

- Mobil / tablet / desktop uyumu olmalıdır.
- WCAG kriterleri (**WCAG (Web Content Accessibility Guidelines)**, web içeriklerinin **engelli bireyler dâhil herkes tarafından erişilebilir** olmasını sağlamak için W3C tarafından yayımlanan **uluslararası erişilebilirlik standartlarıdır**.) desteklenmelidir.
- Klavye ile kullanım mümkün olmalıdır.
- Renk körlüğü uyumu (**Renk körlüğü ile uyum (color-blind accessibility)**, bir web sitesindeki bilgilerin **sadece renge bağlı kalmadan**, renk görme farklılıkları olan kullanıcılar tarafından da **ayırt edilebilir ve anlaşılabilir** biçimde sunulmasıdır. Bu konu **WCAG erişilebilirlik standartlarının** doğrudan bir parçasıdır.) sağlanmalıdır.
- Resimler için alt metinler verilmelidir.

- Geniş beyaz alan içeren temiz bir görsel düzen kullanılmalıdır (zayıf görme yeteneğine karşı)
- Açıklayıcı bağlantı metinleri kullanın (daha fazla bilgi olmadan "burayı tıklayın" ifadesini kullanmaktan kaçının)
- Bağlantıları belirtmek için yalnızca renge güvenmeyin (zayıf renk algısı yeteneğine karşı)
- Ekran titremesi, ekran kayması yapmayın (zayıf görme yeteneğine karşı)
- Sade, anlaşılır dil kullanın (kokteyl partisi etkisine karşı)
- Görsel işaretleme kullanmayın (örneğin, metni kalın yapmak için b yerine strong kullanın; italik yapmak için i yerine em kullanın)
- Görsel biçimlendirme için başlık etiketleri kullanmayın
- Ana gezinme ve sayfa içeriğine "atla" bağlantıları ekleyin
- PDF dosyaları kullanılıyorsa, bilgi için alternatif biçimler sağlayın (HTML, Excel, Word)

Ölçüm Soruları

- Mobilde masaüstü kadar iyi mi?
- Engelli kullanıcılar erişebiliyor mu?

2.6.6. İçerik & SEO Uyum

İçerik hem kullanıcıya hem arama motoruna uygun mu?

- Başlık hiyerarşisi (H1-H6)
- Metin kalitesi
- Anahtar kelime dengesi
- Meta etiketler
- Yapılandırılmış veri

Ölçüm Soruları

- İçerik gerçekten okunuyor mu?
- Arama motoru anlayabiliyor mu?

2.6.7. Genel tasarım düzeltmeleri

- Metin ve arka plan arasında yüksek kontrast kullanın (ters renkler)
- Çok fazla boşluk kullanın
- Daha büyük yazı tipleri kullanın
- Tuş gezinme düğmelerini sol üste koyun

- Bir web sayfasındaki öğeleri ayırt etmek için yalnızca renge güvenmeyin
- Yoğun grafiklerden kaçının
- Sayfa gürültüsünü sınırlayın

3. HTML5

3.1. Giriş

HTML (Hyper Text Markup Language), web sayfaları oluşturmak için kullanılan biçimlendirme dilidir. HMTL5, HTML'in son sürümüdür. İçeriğin işaretlenmesinde etiketler (tags) kullanılır. Her bir etiketin özellikleri (properties) bulunur.

3.2. HTML doküman yapısı

<!DOCTYPE> doküman türünü verir. HTML dokümanının en başında yazılır. Bir HTML dokümanı <html>…</html> etiketleri arasında <head> ve <body> şeklinde iki temel bloktan oluşur. Önce <html> yazılır, peşinden <head>...</head> ve <body>...</body> ve en sonda</html> yazılır. <head>…</head> bloğu sayfayla ilgili ayarları, harici kaynaklarla bağlantıları ve scriptleri içerir. <title> etiketi sayfa başlığını verir. <body>…</body> web sayfasında görüntülenecek içeriği tutar.

3.3. HTML'de açıklama/yorum satırları

HTML satırları arasına açıklamalar eklemek için kullanılır. Bir HTML sayfasının ne amaçla üretildiği de açıklama satırları ile verilebilir. Ayrıca tek satırlı veya çok satırlı yorumlar yazılabilir.

```
<!-- Bu bir HTML açıklaması satırıdır -->
<!--
Sayfa Hidayet Takcı tarafından üretilmiştir!
Sayfanın amacı basit HTML örnekleri oluşturmaktır.
-->
```

3.4. Başlık etiketleri (headings)

Görsel tasarıma ve kullanılabilirliğe hizmet eden unsurlardan birisi başlık etiketleridir. Başlık etiketleri çeşitli boyutlarda başlıkları sunmak için kullanılır. HTML kapsamında Başlık 1'den Başlık 6'ya kadar altı ayrı başlık etiketi bulunur.

```
    <h1>Başlık 1</h1>
    <h2>Başlık 2</h2>
    <h3>Başlık 3</h3>
    <h4>Başlık 4</h4>
    <h5>Başlık 5</h5>
    <h6>Başlık 6</h6>
```

Başlık 1 en büyük başlığı sunarken Başlık 6 en küçük başlığı sunar.

3.5. Metin etiketleri

Metinler konusunda çok sayıda HTML etiketi bulunur. Bunlardan kimisi metnin yapısı kimisi de metnin içeriği üzerindeki düzeltmeleri yapmak için kullanılır. Metin yapısı konusunda paragraf, önceden tanımlı paragraf, satır sonu ve yatay çizgi etiketleri bulunur. Metin içi biçimlendirme kapsamında ise; vurgulama, belirginleştirme, altını çizme, alt simge ve metnin üstünü çizme gibi biçimlendirme etiketleri bulunur.

3.5.1. Paragraf etiketi

Paragraflar en temel HTML öğesidir. Kelime işleme programlarındaki paragraf mantığı HTML dokümanları için de geçerlidir. Paragraf aşağıdaki gibi çalışır.

```
<p> Bir paragraf tanımlar </p>
```

Paragraf olmadan yazılan metinler birbirine bitişik olarak görünürken paragraf içine alınan metinler diğer metinlerden ayrışır.

3.5.2. Önceden tanımlı paragraf etiketi

Şiir gibi girintiler, boşluklar içeren metinler <pre> ile sunulur.

```
<pre> Bu          metin
yazıldığı gibi
     görünecektir. </pre>
```

3.5.3. Satır sonu etiketi

Bir metnin bir sonraki satırdan itibaren başlamamasını istersek <br/> etiketini kullanırız.

```
Birinci satır </br>
İkinci satır </br>
```

3.5.4. Yatay çizgi etiketi

Bir web sayfasının okunabilirliğini artırmak ve bölümler arasında yatay bir çizgi eklemek istersek <hr> etiketi kullanılırız.

```
     <hr> Yatay bir çizgi ekler
```

3.5.5. Vurgulama (Highlighting) etiketi

Diğer metinlere göre daha belirgin metinler oluşturmak için kullanılır.

```
<mark> öğesi HTML5'te yenidir ve bir belgedeki metni işaretlemek
veya vurgulamak için kullanılır.
<p> Bu bir <mark>vurgulu metin</mark> örneğidir </p>
```

3.5.6. Kalın, italik, altı çizgi etiketi

```
<b> Kalın yazılmış metin </b>
<i> Eğik yazılmış metin </i>
<u> Altı çizgili metin </u>
```

3.5.7. Kısaltma (abbrevation) etiketi

Bazı ifadeleri kısaltma olarak işaretlemek için <abbr> etiketini kullanılır.

```
<p>     Türkiye     Cumhuriyeti     <abbr     title="Türkiye
Cumhuriyeti">TC</abbr> kısaltması ile verilir. </p>
```

3.5.8. Ekleme, silme, üstünü çizme

```
<ins> Eklenen metin</ins>
<del> Silinmiş olarak etiketlenen metin </del>
<s> Üzeri çizili metin </s>
```

3.5.9. Üst ve alt işaretleri (Superscript and Subscript):

Özellikle bilimsel gösterim yapacağımız zaman alt indis ve üst indis kullanmaya ihtiyaç duyarız. O zaman üst indis için sup, alt indis için sub kullanılır.

```
X <sup>2</sup>
Y <sub>3</sub>
```

3.6. Bağlantılar (links)

Web sayfalarının belki en önemli özelliği sayfaların birbirlerine bağlantılar üzerinden bağlı olmasıdır. Aşağıda bağlantı etiketi ve özellikleri ayrıntılı olarak verilmiştir.

<a>: sayfa içi ya da sayfalar arası hatta siteler arası yönlendirme yapmak için kullanılan etikettir.

- **href**: bağlantının gideceği hedef adresi belirten özelliktir. Değeri http://example.com/ gibi bir web sitesinin URL adresi veya /about-us/ gibi sunucudaki bir dizin ya da index.html gibi sunucudaki bir sayfa olabilir.

```
<a href="http://example.com">Example Sitesi</a>
<a href="/about-us/">Hakkımızda</a>
<a href="index.html">Ana Sayfa</a>
```

- **hreflang**: çok dilli veya çok bölgeli web sitelerinde **arama motorlarına bir sayfanın hangi dil ve/veya ülkeye yönelik olduğunu bildiren HTML özelliğidir**. Özellikle **SEO** açısından kritik öneme sahiptir.

```
<a href="http://www.cumhuriyet.edu.tr"
hreflang="en">S.C.U.</a>
```

- **target**: Bağlantının nerede açılacağını belirtir, Örn. yeni bir sekmede veya bağlantı yapılan sayfada olabilir. Olası değerler şunlardır: **_blank, _self, _parent** vs.

  _blank: bağlantıyı yeni bir sekmede açar.

  _self: bağlantıyı aynı (mevcut) sekmede açar.

  _parent: Eğer sayfa bir **iframe içinde** çalışıyorsa, bağlantıyı **üst (parent) sayfada** açar

```
<a  href="http://ceng1.cumhuriyet.edu.tr"  target="_blank">SCU
Bilmuh </a>
```

- **title**: Bir bağlantı hakkında ek bilgi belirtir. İmleç bağlantı üzerinde hareket ettiğinde bilgi genellikle ipucu metni olarak gösterilir.

```
<a      href="http://ceng1.cumhuriyet.edu.tr"      title="örnek
başlık">Örnek başlık</a>
```

- **download**: Bir kullanıcı bir bağlantıyı tıkladığında browser üzerinde açmak yerine hedefi indirmeye çalışır. Özniteliğin değeri, indirilen dosyanın adı olacaktır.

```
<a href="/images/ornek.jpg" download> Resmi indir</a>
```

- **Sayfa içi bağlantı**: Bağlantı sayfa içinde verilmek istendiğinde aşağıdaki kullanım tercih edilir.

```
<h2 id="Konu1">İlk konu</h2>
<p>İlk konu hakkındaki içerik </p>
<h2 id="Konu2">İkinci konu</h2>
<p>İkinci konu hakkındaki içerik </p>
<h1>İçindekiler </h1>
<a href='#Konu1'>İlk konu için bağlantı…</a>
<a href='#Konu2'>İkinci konu için bağlantı…</a>
```

- **Aynı sitedeki bir alt klasöre bağlantı verme:**

```
<a href="/example">example alt klasörüne bağlantı</a>
<a  href="/example/page">example  altındaki  page  klasörüne
bağlantı</a>
```

- **Bir telefon numarasına bağlantı verme:**

```
<a href="tel:11234567890">Bizi arayın</a>
```

- **Çalışan bir script için bağlantı verme:**

```
<a href="javascript:myFunction();">Kodu çalıştır</a>
```

```
<a  href="#"  onclick="myFunction();  return  false;">Kodu
çalıştır</a>
```

- **Elektronik posta için bağlantı örnekleri:**

```
<a href="mailto:example@example.com">Send email</a>
<a
href="mailto:example@example.com?cc=john@example.com&bcc=jan
e@example.com">Send email</a>
<a
href="mailto:example@example.com?subject=Example+subject&bod
y=Message+text">Send email</a>
```

3.7. Dosyayı bağlantı ile dâhil etme

<link>: bir sayfaya başka bir dosyayı dâhil etmek için kullanılan etikettir.

- **rel**: bağlantı kurulacak dosyanın türünü verir.
- **href**: bağlantı kurulacak dosyanın adını veren özelliktir.

```
<link rel="stylesheet" href="styles.css">
```

Bu yazımla aynı klasörde bulunan *"styles.css"* dosyası mevcut sayfaya dahil edilir.

3.8. Listeler

HTML, listeleri belirtmek için üç yol sunar: **sıralı listeler (ol)**, **sırasız listeler (ul)** ve **açıklama listeleri (dl)**.

- Sıralı listeler, liste öğelerinin sırasını belirtmek için sıra dizilerini kullanır,
- Sırasız listeler, öğeleri belirli bir sırada listelemek için madde işareti sembolü kullanır,
- Açıklama listeleri, öğeleri alt öğeleriyle birlikte listelemek için girintiler kullanır.

Sıralı listeler:

```
<ol>
<li>Eleman</li>
<li>Diğer eleman</li>
<li>Bir diğer eleman</li>
</ol>
```

Sırasız listeler:

```
<ul>
<li>Item</li>
<li>Another Item</li>
<li>Yet Another Item</li>
</ul>
```

Açıklama listeleri:

```
<dl>
<dt>Değer 1</dt>
<dd>Değer 1 için açıklama</dd>
<dt>Değer 2</dt>
<dd>Değer 2 için açıklama</dd>
</dl>
```

Liste elemanlarının altında onlara ait açıklamalar da bulunur.

İç içe listeler:

```
<ul>
<li>item 1</li>
<li>item 2
<ul>
<li>sub-item 2.1</li>
<li>sub-item 2.2</li>
</ul>
</li>
<li>item 3</li>
</ul>
```

3.9. Tablolar

HTML <table> öğesi, verilerin (metin, resimler, bağlantılar, diğer tablolar vb.) satırlar ve sütunlar içeren iki boyutlu bir tabloda görüntülemesine olanak tanır.

- <tr> satır oluşturmak için
- <th> başlık bölümünde sütun oluşturmak için
- <td> içerik bölümünde sütun oluşturmak için kullanılır.

- <th> ve <td> etiketlerinin her ikisi de sütun oluşturmakla birlikte çıktıları ve görevleri farklıdır.

```
<table>
  <tr>
    <th>Heading 1/Column 1</th>
    <th>Heading 2/Column 2</th>
  </tr>
  <tr>
    <td>Row 1 Data Column 1</td>
    <td>Row 1 Data Column 2</td>
  </tr>
  <tr>
    <td>Row 2 Data Column 1</td>
    <td>Row 2 Data Column 2</td>
  </tr>
</table>
```

3.9.1. Colspan özelliği

Bir tablonun iki sütununu birleştirmek istediğimizde **colspan** özelliğini kullanırız. Yandaki çıktıyı verecek satırlar aşağıdaki gibidir.

```
<table>
<tr>
     <td>1</td>
     <td colspan="2">2</td>
</tr>
<tr>
     <td>4</td>
     <td >5</td>
     <td>6</td>
</tr>
</table>
```

Çıktı:

1	2	
4	5	6

3.9.2. Rowspan özelliği

Bir tablonun iki satırını birleştirmek istediğimizde rowspan özelliğini kullanırız. Yandaki çıktıyı verecek satırlar aşağıdaki gibidir.

```
<table>
<tr>
     <td>1</td>
     <td rowspan="2">2</td>
     <td>3</td>
</tr>
<tr>
     <td>4</td>
     <td>6</td>
</tr>
</table>
```

Çıktı:

1	2	3
4		6

3.10. Resimler

Bir sayfaya resim eklemek için <img> etiketi kullanılır.

```
<img src="images/logo.png">
```

Parametreler:

- **src**: görüntülenecek resmin adresini belirler // bir nevi href gibi çalışır
- **srcset**: farklı çözünürlükler için farklı resimler sağlar // yüksek çözünürlüklü, küçük monitörler vs. Farklı ekran boyutlarına göre resim görünmesi için aşağıdaki kod kullanılabilir. Öncelikle aynı resim farklı boyutlarda kaydedilmiş olmalıdır. (w değeri tarayıcıya resim genişliğini söyler).

```
<img
  src="foto-800.jpg"
  srcset="
```

```
    foto-400.jpg 400w,
    foto-800.jpg 800w,
    foto-1600.jpg 1600w
  "
  alt="Doğa fotoğrafı"
>
```

w değeri px ile aynı anlama gelir. img srcset ile kullanılır.

- **usemap**: Resmin belirli bölgelerini tıklanabilir hale getirir. Resim ile resim haritası arasında bağ kurmak için usemap kullanılır. Resim haritasında bağlantılar bulunur.

```
<img src="harita.png" alt="Türkiye haritası"
usemap="#turkiye">
<map name="turkiye">
<area shape="rect" coords="50,50,150,150" href="ankara.html"
alt="Ankara">
<area shape="circle" coords="300,200,40" href="istanbul.html"
alt="İstanbul"></map>
```

- **alt**: herhangi bir nedenle resim görüntülenemezse onun yerine alternatif metin sağlar.

```
<img src="urun.jpg" alt="Kırmızı spor ayakkabı">
```

- **width**: resim genişliğini sunar.
- **height**: resim yüksekliğini sunar.

```
<img src="manzara.jpg" alt="Dağ manzarası" width="300" height="200">
```

***Bir tavsiye**: resimleri orijinal boyutlar haricinde büyütecek ya da küçültecekseniz ya sadece **width** değerini ya da sadece **height** değerini güncelleyiniz. Aksi takdirde basık ya da uzun resimler görünecektir.*

3.11. Formlar

İstemci sunucu mimarisinin en önemli fonksiyonlarından birisi istemci tarafından gönderilen veri veya isteğin sunucu tarafından karşılanmasıdır. İstemciden sunucuya veri gönderilmesinde kullanılan arabirimler Formlar olarak bilinir.

3.11.1. Form elemanları

3.11.1.1. Text

Sunucuya gönderilmek üzere metin verileri girmek için kullanılır.

```
<input type="text">
```

Çıktı:

```
<input type="text" size="50">
```

Çıktı:

3.11.1.2. Checkbox ve radio buttons

Onay kutusu ve radyo butonlar için kullanılır.

```
<input type="radio" name="color" id="red">
<input type="radio" name="color" id="green">
<input type="radio" name="color" id="blue">
<input type="checkbox" name="color" id="red">
```

Çıktı:

3.11.1.3. Color

Renk bilgisi girişi için kullanılır.

```
<input type="color" name="favcolor" value="#ff0000">
```

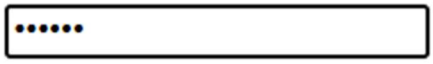
Çıktı:

3.11.1.4. Password

Parola bilgisi girişi için kullanılır.

```
<input type="password" name="password" placeholder="Password">
```

Çıktı:



3.11.1.5. File

Form üzerinden karşıya dosya gönderimi için kullanılır. Asıl aktarım işini sunucu tarafındaki script yerine getirir.

```
<input type="file" name="fileSubmission" id="fileSubmission">
```

Çıktı:

Dosya Seç Dosya seçilmedi

3.11.1.6. Button

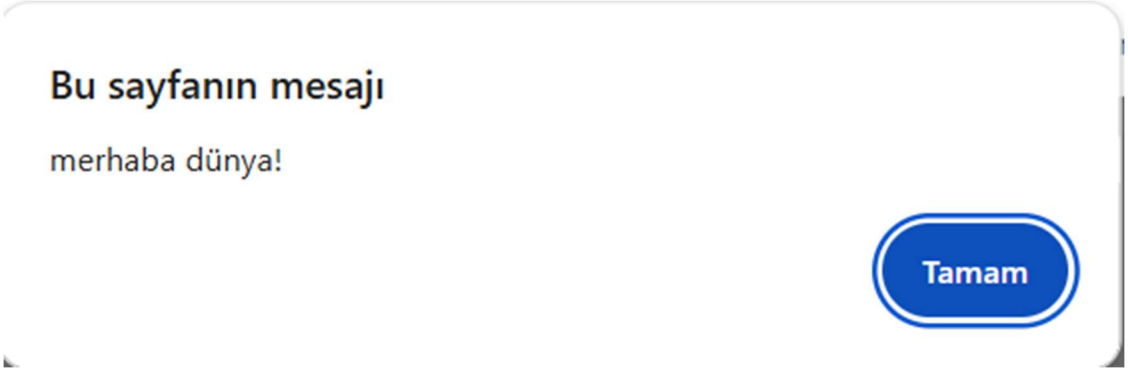
Bir fonksiyonu çalıştırmak için kullanılır.

```
<input type="button" onclick="alert('merhaba dünya!')" value="message">
```

Çıktı:

message

Message butonu oluşturulur. Bu butona tıklayınca aşağıdaki çıktı (alert) elde edilir.



3.11.1.7. Submit

Verileri sunucuya göndermek için kullanılır.

```
<input type="submit" value="Gönder">
```

Çıktı:

Gönder

3.11.1.8. Reset

Gönderimi iptal etmek için kullanılır.

```
<input type="reset" value="Vazgeç">
```

Çıktı:

Vazgeç

3.11.1.9. Hidden

Sunucuya gönderilecek ama sayfada gözükmeyecek bilgiler için kullanılır.

```
<input type="hidden" name="inputName" value="inputValue">
```

3.11.1.10. Tel

Telefon bilgisi girişi için kullanılır.

```
<input type="tel" value="+8400000000">
```

Çıktı:

3.11.1.11. Email

E-posta bilgisi girişi için kullanılır.

```
<input type="email" name="email">
```

Çıktı:

3.11.1.12. Number (Sayı)

Sayı türündeki verileri girmek için kullanılır.

```
<input type="number" id="quantity" name="miktar" min="1"
max="5">
```

Çıktı:

3.11.1.13. Range (Aralık)

Belli bir aralık içerisinde sayı girişi için kullanılır.

```
<input type="range" id="points" name="nokta" min="0" max="10">
```

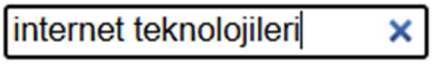
Çıktı:

3.11.1.14. Search (Arama)

Arama metin kutusu eklemek için kullanılır.

```
<input type="search" name="googlesearch">
```

Çıktı:



3.11.1.15. Image (Resim)

Form üzerinden resim girişi için kullanılır.

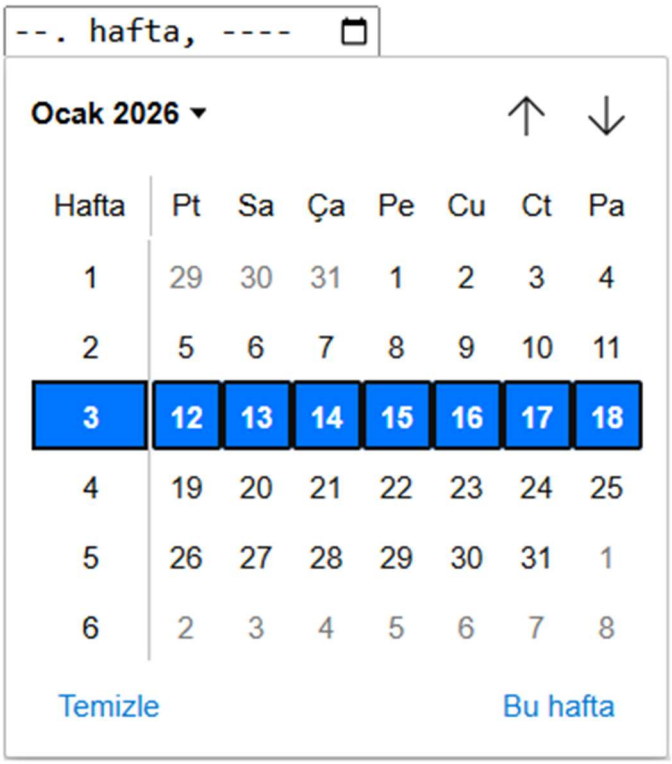
```
<input type="image" src="img.png" height="50px" width="50px"/>
```

3.11.1.16. Week (Hafta)

Hafta bilgisi girişi için kullanılır.

```
<input type="week" />
```

Çıktı:

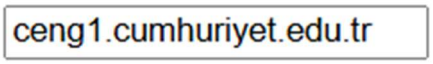


3.11.1.17. URL

Web adresi bilgisi girişi için kullanılır.

```
<input type="url" name="Homepage">
```

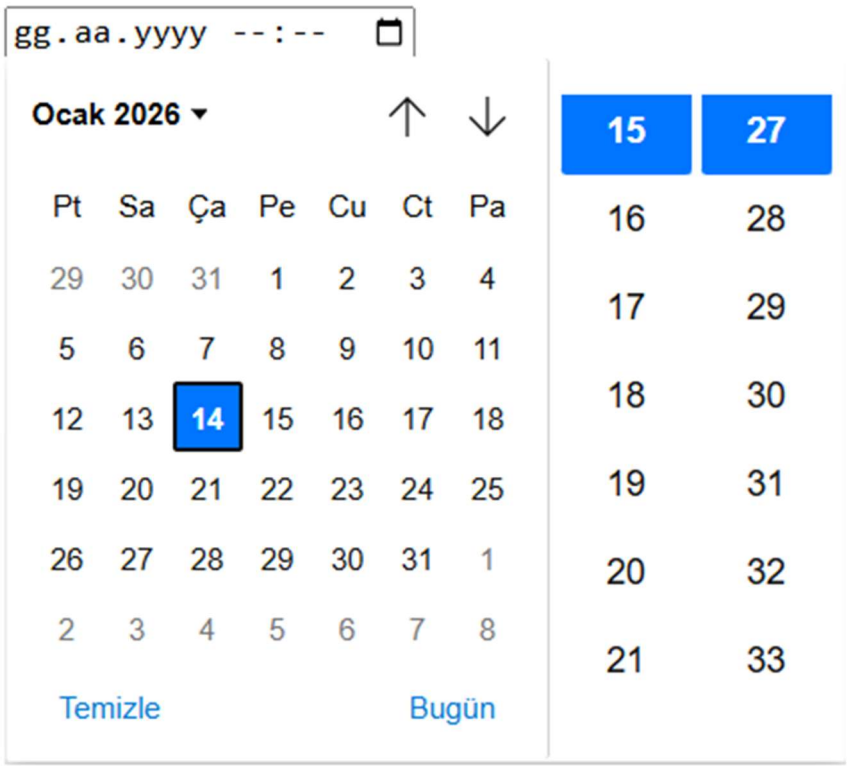
Çıktı:



3.11.1.18. Date-time local

```
<input type="datetime-local" />
```

Çıktı:

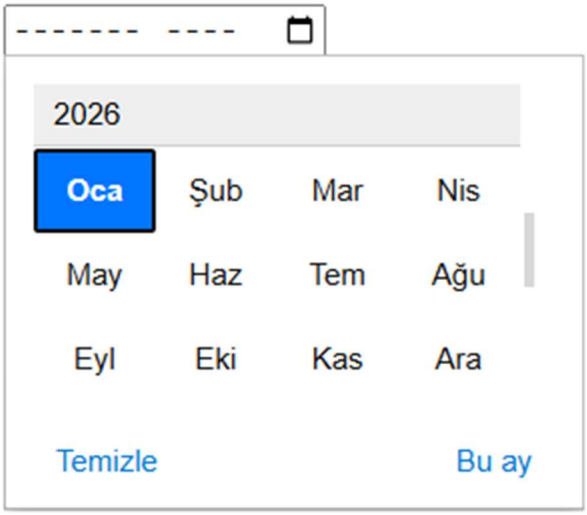


3.11.1.19. Month

Ay bilgisini seçmek için kullanılır.

```
<input type="month" />
```

Çıktı:

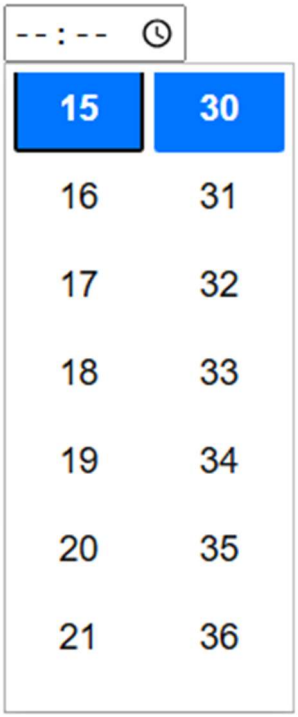


3.11.1.20. Time

Zaman bilgisini seçmek için kullanılır.

```
<input type="time" />
```

Çıktı:

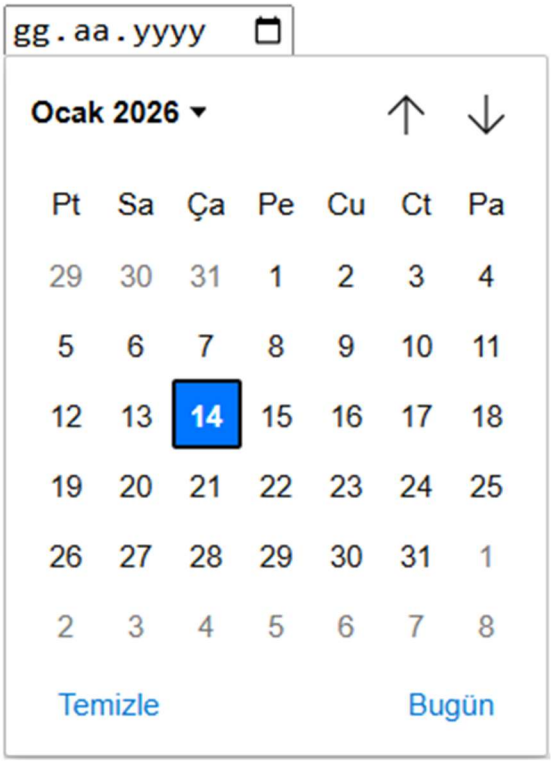


3.11.1.21. Date

Tarih bilgisi seçimi için kullanılır.

```
<input type="date" />
```

Çıktı:



3.11.2. Giriş Doğrulama

Form üzerinden veri girişi yaparken bazı kısıtlara uymak gerekebilir. Böylesi durumlarda giriş doğrulama yapılır. Sayı türünde ve verilen sınırlar içinde veri girişi için aşağıdaki kod yazılabilir.

```
Marks: <input type="number" size="6" name="puan" min="0" max="100" />
Sayı: <input pattern="\d*" title="Sadece sayı lütfen.">
```

Puan adındaki number türündeki alana minimum 0 maksimum 100 değeri girilebilir, veri giriş alanı boyutu 6 birimdir. Ayrıca sayı başlıklı girişin sadece sayı değerler içermesi beklenmektedir.

Boş geçilemeyecek alanlar için required özelliği kullanılır.

```
<input type="text" name="name" required>
<input type="email" name="email" required>
```

Formnovalidate giriş doğrulamayı devre dışı bırakmak için kullanılır. Aksi takdirde formun içindeki ögeler giriş doğrulamasından geçmeden işleme alınmaz.

```
<form>
<input type="text" name="name" required>
<input type="email" name="email" required>
<input pattern="\d*" name="number" required>
<input type="submit" value="Publish">
<input type="submit" value="Save" formnovalidate>
</form>
```

Formnovalidate yazılmadığında name, email ve number alanları için geçerli olan giriş doğrulama formnovalidate sonrası geçersiz kalacaktır.

3.11.3. Form gönderme

Submitting:

```
<form action="action.php" method="get">
<form action="action.php" method="post">
```

Form üzerinden gönderilecek veriler get veya post metodundan biri yardımıyla sunucuya gönderilir. Ayrıca sunucu üzerinde action.php ile işleme yapılır.

Target özelliği form başlığında:

```
      <form target="_blank">
```

Sunucu scriptinin boş bir sayfada açılmasını sağlar.

3.11.4. Form elemanlarının gruplanması

Form üzerindeki veri girişinde birbiriyle daha ilişkili alanlar kutular şeklinde diğerlerinden ayrılır. Bunun için fieldset kullanılır.

```
<form>
<fieldset>
<legend> Birinci veri alanı:</legend>
     Alan 1:<br>
     <input type="text"><br>
     Alan 2:<br>
     <input type="text"><br>
</fieldset><br>
<fieldset>
<legend>İkinci veri alanı:</legend>
     Alan 3:<br>
```

```
     <input type="text"><br>
     Alan 4:<br>
     <input type="text"><br>
</fieldset><br>
<input type="submit" value="Gönder">
</form>
```

3.12. Div etiketi

HTML sayfalarını bir bütün olarak değil de ayrılabilir bölümler olarak ele almak onu düzenlemek için en akılcı yöntemdir. Bunun için div etiketi kullanılır. Division (bölümleme) kelimesine div adını alan bu etiket HTML öğelerini bir araya getirmek ve onlara stiller uygulamak için kullanılan bir kapsayıcıdır. Herhangi bir anlamsal değeri yoktur ve yalnızca düzen amacıyla kullanılır.

```
<div>
<h1> div örneği </h1>
<p> Selam!  Bu bir paragraftır </p>
</div>
div {
display: block;
}
```

3.13. HTML ve CSS

Web sayfalarının yapısını HTML, biçimini CSS ayarlar. Satır içi yazım şeklinde de kullanılabilen CSS için sıklıkla dâhili ve harici style ekleme yaygındır. Daha metodolojik olan ve web sitesinin tamamında geçerli olan CSS stilleri tanımlamak için yapılması gereken stilleri stylesheet.css gibi bir dosyada toplamak ve onu HTML dokümanına dahil etmektir. Harici stil dosyası ekleme aşağıdaki gibi yapılır.

```
<head>
<link rel="stylesheet" type="text/css" href="stylesheet.css">
</head>
```

Kimi zaman sadece ilgili sayfada geçerli olan stiller de eklenebilir. Sayfanın head bölümünde yer alan stil ekleme aşağıdaki gibi yapılır.

```
<head>
<style type="text/css">
body {
background-color: gray;
}
</style>
</head>
```

Satır içi stil ekleme ise basitçe aşağıdaki gibi yapılır:

```
<span style="color: red">This text will appear in red.</span>
<p style="text-align:center;">Ortalı metin </p>
```

3.14. HTML ve Javascript

HTML yapı, CSS biçim sağlarken Javascript etkileşim sağlar. Bir HTML sayfasına doğrudan javascript kodu eklemek istersek aşağıdaki satırları yazabiliriz.

```
     <script>
     // JavaScript code
     </script>
```

Harici olarak oluşturulmuş ve example.js şeklinde bir dosyaya kaydedilmiş javascript kodlarını HTML dokümanımıza eklemek istersek aşağıdaki satırı yazarız:

```
     <script src="example.js"></script>
```

Eğer javascript kodunu disable yapmak istersek o zaman aşağıdaki kodu yazarız:

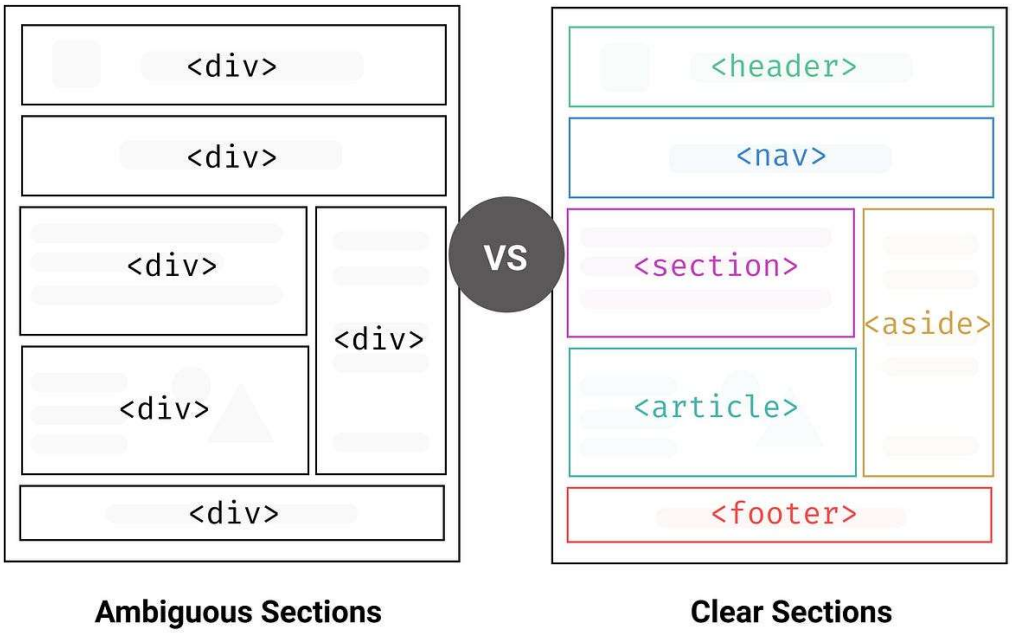
```
     <script>
     document.write("Merhaba dünya!");
     </script>
     <noscript>Bu browser Javascript desteklemez.</noscript>
```

3.15. Semantik elemanlar

Bir semantik element, hem insanlar hem de makineler tarafından okunabilir bir elementtir. Semantik olmayan elementler ise sadece insanlar tarafından okunabilir.

- Semantik olmayan öğelere örnekler: ve - element içeriğin ne olduğu hakkında bir şey söylemez.
- Semantik öğelere örnekler: <img>, <table> ve <article> - element içeriğin ne olduğu hakkında bilgi vericidir.

Semantik elementler görsel değil anlamsal ögelerdir.



Yukarıdaki görselde olduğu gibi solda birçok bölüm var fakat her birinin fonksiyonu hakkında bir bilgiye sahip değiliz. Bununla birlikte sağdaki bölümlerin ne amaca hizmet ettiği açıktır. Yapılan işin içeriğini de açıklayan bölümler tanımlamaya imkân sağlayan yapılar semantik elementlerdir.

HTML'de bir web sayfasının farklı bölümlerini tanımlamak için kullanılabilecek çeşitli semantik elemenler vardır:

3.15.1. <nav> element

Gezinme bağlantılarını içerir. Menü, sayfa içi linkler vb.

```
<nav>
  <a href="#anasayfa">Anasayfa</a>
```

```
  <a href="#hakkimizda">Hakkımızda</a>
  <a href="#iletisim">İletişim</a>
</nav>
```

HTML5 öncesi navigasyon için sıklıkla listeler kullanılırdı. Klasik kullanım örneği aşağıdaki gibidir.

```
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">About</a></li>
<li><a href="#">Contact</a></li>
</ul>
```

3.15.2. <article> element

Bağımsız ve tek başına anlamlı içerik için kullanılır (makale, haber, blog yazısı).

```
<article>
  <h2>Yapay Zekâ Nedir?</h2>
  <p>Yapay zekâ, insan benzeri öğrenme yeteneği olan sistemlerdir.</p>
</article>
```

Bir <article> kendi başına anlamlı olmalı ve web sitesinin geri kalanından bağımsız olarak da sunulabilmelidir. <article> öğesinin kullanılabileceği örnekler:

- Forum gönderileri
- Blog gönderileri
- Kullanıcı yorumları
- Ürün kartları
- Gazete makaleleri

3.15.3. <section> element

Sayfa içinde **mantıksal olarak gruplanmış içerik** bölümlerini temsil eder.

```
<section>
  <h2>Hizmetlerimiz</h2>
```

```
  <p>Yazılım, eğitim ve danışmanlık hizmetleri sunuyoruz.</p>
</section>
```

Section örnekleri:

- Kitap bölümleri
- Giriş bölümü
- Yeni ögeler
- İletişim bilgisi

<article> öğeleri içeren <section> öğeleri veya <section> öğeleri içeren <article> öğeleri yazılabilir. Çünkü bir <section> içerisinde bir <header> ve bir <article> olabilir. Benzer şekilde bir <article> ögesinin <section> ile ifade edilebilecek alt bölümleri olabilir.

3.15.4. <aside> element

Ana içerikle **dolaylı olarak ilişkili** yan içerikler (reklam, ek bilgi, notlar).

```
<aside>
  <h3>Not</h3>
  <p>Bu makale HTML5 semantik etiketlerini anlatır.</p>
</aside>
```

3.15.5. <main> element

Sayfanın **en önemli ve benzersiz ana içeriğini** kapsar. Bir sayfada **sadece bir tane** olmalıdır.

```
<main>
  <h1>Web Programlama</h1>
  <p>HTML, CSS ve JavaScript konuları işlenmektedir.</p>
</main>
```

<main> öğesi, web sayfasının ana içeriğini sunar.

Üstbilgiler, altbilgiler, gezinme, logolar vb. gibi yinelenen içerik, öğenin dışına yerleştirilir.

```
<body>
<header> <nav> Navigasyon buraya </nav> </header>
<main>
<h1> Blog gönderisi </h1>
```

```
<p> Blog gönderisi için giriş </p>
<article> <h2> Kaynaklar </h2> </article>
<article> <h2> Yorumlar </h2> </article>
</main>
<footer>...</footer>
</body>
```

3.15.6. <header> element

Bir sayfa veya bölüm için **başlık, logo, menü** gibi üst bilgileri içerir.

```
<header>
  <h1>BilMuh Web Sitesi</h1>
  <p>Bilgisayar Mühendisliği Bölümü</p>
</header>
```

<header> öğesi, en yakın ata bölümleme içeriği veya bölümleme kök öğesi için giriş içeriğini temsil eder. <header> elemanı <body> içerisinde, <main> içerisinde, <section> içerisinde hatta <article> içerisinde geçebilir.

3.15.7. <footer> element

Sayfa veya bölümün **alt bilgilerini** içerir (telif, iletişim, bağlantılar).

```
<footer>
  <p>© 2026 SCÜ Bilgisayar Mühendisliği</p>
</footer>
```

Sayfanın sonunda yer alır. Sayfada bir kere yer alır.

3.15.8. <figure> element

Resim, grafik, diyagram gibi **bağımsız görsel içerikleri** sarmalar.

```
<figure>
  <img src="grafik.png" alt="Başarı grafiği">
</figure>
```

3.15.9. <figcaption> element

<figure> içindeki görsel için **açıklama** ekler.

```
<figure>
  <img src="ai.png" alt="Yapay zekâ">
  <figcaption>Şekil 1: Yapay zekâ sistemlerinin genel yapısı</figcaption>
</figure>
```

3.15.10. <summary> element

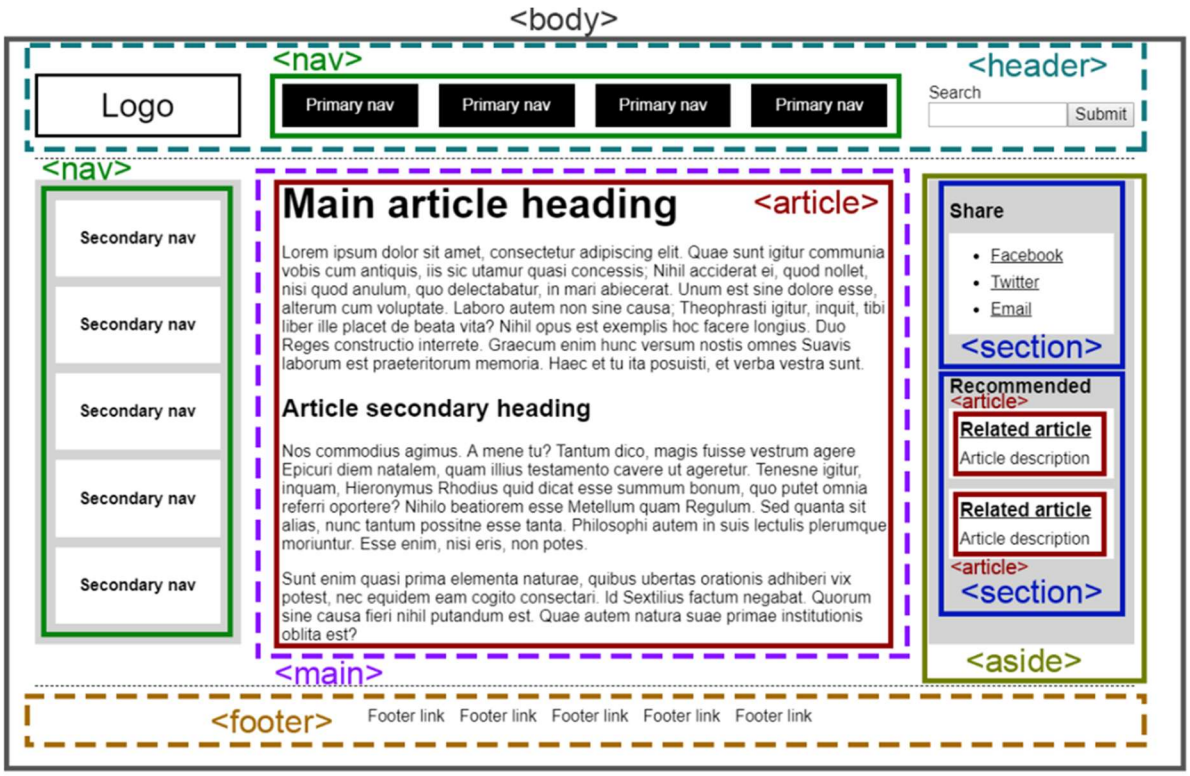
<details> etiketi içinde **kısa özet / tıklanabilir başlık** görevi görür.

```
<details>
  <summary>Daha fazla bilgi</summary>
  <p>HTML5 semantik etiketler SEO ve erişilebilirliği artırır.</p>
</details>
```

Önce Daha fazla bilgi yazan bir ok ve açılınca da aşağıdaki gibi bir çıktı elde edilir.

▼ Daha fazla bilgi

HTML5 semantik etiketler SEO ve erişilebilirliği artırır.



Semantik elementlerin bir arada örneklenmesi

3.15.11. <label> element

Label element, bir form öğesine hangi bilginin girileceğini belirtir. Girdi ile sadece görsel bir birliktelik değil aynı zamanda programatik bir birliktelik sağlar. Label etiketi tıklandığında imleç onunla ilişkili metin kutusuna konumlanır.

```
<form action="/login" method="POST">
<label for="username">Username:</label>
<input id="username" type="text" name="username" />
<label for="pass">Password:</label>
<input id="pass" type="password" name="pass" />
<input type="submit" name="submit" />
</form>
```

3.15.12. <output> element

Output element form üzerinde ve giriş sayfasında işlemler yapmak için kullanılır.

```
<form id="form1" oninput="out1.value = parseInt(in1.value, 10)
* parseInt(in2.value,10)">
```

```
<fieldset>
<legend>Out örneği</legend>
     <input type="number" id="in1" value="0"> <br/>
     <input type="number" id="in2" value="0">
</fieldset>
</form>
 <output id="out1" for="in1 in2" form="form1">0</output>
```

parseInt: aldığı string değeri verilen tabana göre sayısal değere dönüştürür.

oninput: veri girişi yapılırken işlem tetiklenir ve sonuç out1 nesnesinde görünür.

3.16. Ses kontrolleri

Ses kontrolleri sayesinde web sayfası üzerinden .mp3 gibi ses dosyalarını çaldırabilirsiniz.

```
<audio controls>
<source src="file.mp3" type="audio/mpeg">
Browser audio elemanlarını desteklemiyor.
</audio>
```

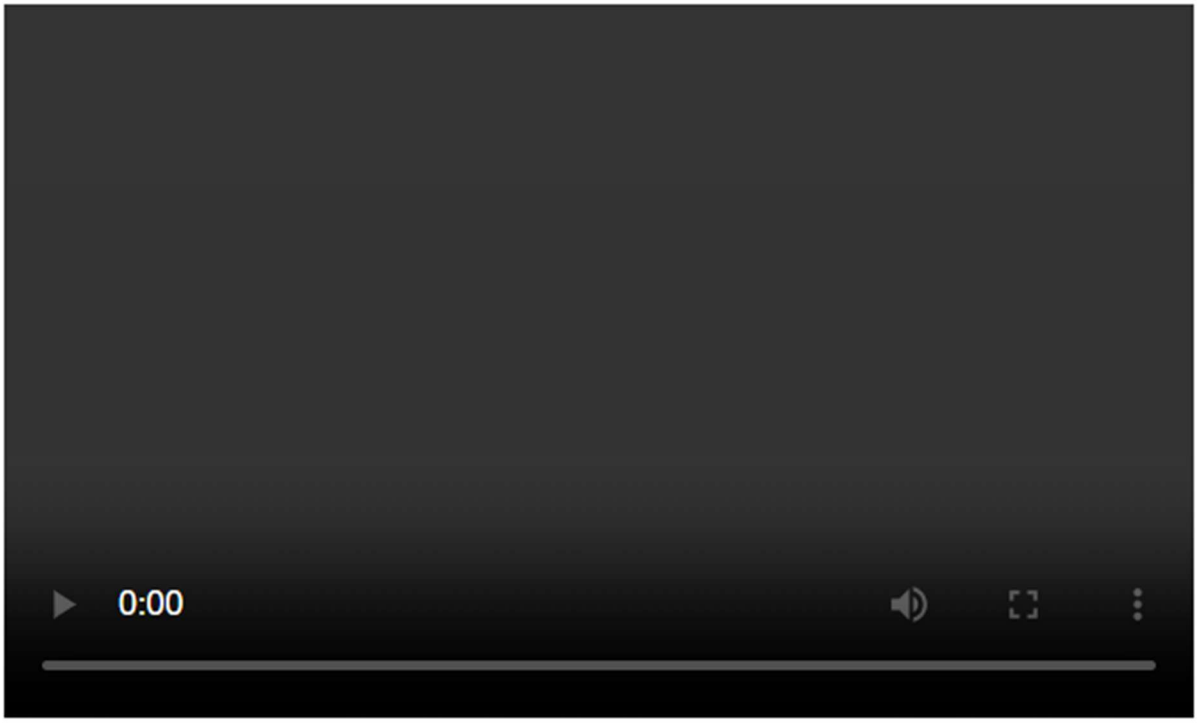
Çıktı:

3.17. Video kontrolleri

Video kontrolleri sayesinde web sayfası üzerinden .mp4 gibi ses dosyalarını çaldırabilirsiniz.

```
<p> HTML Bölüm 3 </p>
<video width="500" height="300" controls>
<source src="ders5.mp4" type="video/mp4">
Browser video etiketini desteklemiyor.
</video>
```

Çıktı:



3.18. Seçim (select) kontrolleri

Seçim yapmanın birçok yöntemi vardır. Uzayan listeden tek bir öge seçmek için bir örnek aşağıda verilmiştir.

```
<select name="">
<option value="1">Bir</option>
<option value="2">İki</option>
<option value="3">Üç</option>
</select>
```

Çıktı:

Eğer tek bir öge değil de birden çok öge seçmek istiyorsanız aşağıdaki gibi bir kod parçası yazabilirsiniz.

```
<select name="" multiple>
<option value="1">Bir</option>
<option value="2">İki</option>
<option value="3">Üç</option>
</select>
```

Çıktı:

Gruplandırılmış elemanların seçimi bir başka seçme yöntemidir. Örnek aşağıda verilmiştir.

```
<select name="">
<option value="süt">Süt</option>
<optgroup label="Meyveler">
<option value="muz">Muz</option>
<option value="çilek">Çilek</option>
</optgroup>
<optgroup label="Sebzeler" disabled>
<option value="havuç">Havuç</option>
<option value="kabak">Kabak</option>
</optgroup>
</select>
```

Çıktı:

3.19. Datalist element

Bir <datalist> elementi, bir <input> öğesi için önceden tanımlanmış seçeneklerin bir listesini belirtir. <input> öğelerinde "otomatik tamamlama" özelliği sağlar. Kullanıcılar yazarken bir açılır seçenekler listesi göreceklerdir.

```
<input list="diller">
<datalist id="diller">
<option value="PHP">
<option value="Perl">
<option value="Python">
<option value="Ruby">
<option value="C++">
</datalist>
```

Çıktı:

3.20. Embed element

Embed element HTML5'te yenidir. Bu öğe, harici (genellikle HTML olmayan) bir uygulama veya etkileşimli içerik için bir entegrasyon noktası sağlar.

```
<embed src="myflash.swf">
<embed type="video/mp4" src="video.mp4" width="640"
height="480">
```

3.21. IFrames kullanımı

IFrames: Sayfanıza başka bir sayfa eklemek için kullanılabilir. Konteyner olarak görülebilir.

```
    <iframe src="base.html"></iframe>
```

Sandboxing: Güvenilir şekilde başka bir sayfayı eklemek için kullanılır.

```
    <iframe sandbox src="http://example.com/"></iframe>
```

Sayfanın komut dosyalarını çalıştırmasına ve form göndermesine izin vermek için, korumalı alan özelliğine allow-scripts ve allow-forms eklenir.

```
<iframe sandbox="allow-scripts allow-forms"
src="http://example.com/"></iframe>
```

Çerçeve boyutunun ayarlanması:

```
<iframe src="base.html" width="800" height="600"></iframe>
```

Srcdoc özelliğinin kullanımı:

```
<iframe srcdoc="<p>IFrames are cool!</p>"></iframe>
```

Aşağıdaki örnekte different.html, webpage.html ve iframe.html dosyalarını görüyorsunuz.

different.html

```
<!DOCTYPE html>
<html>
	<head>
		<title> different.html </title>
	</head>
	<body>
	different.html dosyasına hoş geldiniz!!!
	</body>
</html>
```

webpage.html

```
<!DOCTYPE html>
<html>
	<head>
		<title> webpage.html </title>
	</head>
	<body>
	webpage.html dosyasına hoş geldiniz!!!
	</body>
</html>
```

iframe.html

```
<!DOCTYPE html>
<html>
	<head>
		<title> iframe.html </title>
	</head>
	<body style="text-align:center;">
	<nav>
		<a href="different.html" target="myIframe">different.html içeriği göster</a> |
		<a href="webpage.html" target="myIframe">webpage.html içeriği göster</a> |
		<a href="../media/audio.html" target="myIframe"> Müzik Dinle </a> |
		<a href="http://ceng1.cumhuriyet.edu.tr" target="myIframe"> SCÜ Bilmuh
</a>
	</nav>
	<iframe src="webpage.html" name="myIframe" width=800 height=600></iframe>
	</body>
</html>
```

Çıktı:

different.html içeriği göster | webpage.html içeriği göster | Müzik Dinle | SCÜ Bilmuh

different.html dosyasına hoş geldiniz!!!

Iframe yapısı sayesinde tek bir sayfa üzerinden birden çok sayfa ve bağlantıya erişim imkânı sağlanabilmiştir. Eski fakat oldukça kullanışlı bir yapısı vardır.

4. CSS

4.1. CSS Nedir?

Basamaklı stil sayfaları (Cascading Style Sheets) web sayfalarının görünümünü kontrol etmek için kullanılacak bir web kodlama teknolojisidir.

- HTML web sayfalarının yapısıyla,
- CSS web sayfalarının görünümü ile ilgilenir.

CSS öğrenmenin amacı daha görsel bir web sayfası ortaya koymaktır.

4.2. CSS yapısı

CSS ile bir elemana (element - etiket) bir biçimlendirme (stil) aşağıdaki gibi uygulanır:

```
element {
property: value;
}
```

<u>Örnek 1</u>: h1 etiketinin font büyüklüğünü 72piksele getirmek için;

```
 h1 {
font-size: 72px;
}
```

<u>Örnek 2</u>: p etiketi kullanılan alanın geri plan rengini mavi yapmak için;

```
p {
background-color:blue;
}
```

Örnek 3: bir yatay çizgi etiketinin rengini mavi, uzunluğunu 480 piksel, yüksekliğini 10 piksel, düzenini ortalı yapacak stili yazınız.

Örnek 4: bir div elemanının uzunluğunu 400 piksel, yüksekliğini 300 piksel, metin rengini mavi, geri plan rengini sarı, yazı fontunu times yapacak stili yazınız.

4.3. Bir elemana (element) birden çok stil özelliği uygulama

Stil oluştururken bir elemana birden fazla biçimlendirme uygulanabilir. Aşağıdaki örnek, daha önce gösterildiği gibi <h1> etiketini biçimlendirmek için kullanılan yazımı göstermektedir:

```
h1 {
border-width: 1px; // kenar genişliği 1 piksel olacak.
border-style: solid; // kenar çizgisi sürekli olacak.
border-color: black; // kenar rengi siyah olacak.
font-size: 72px; // font büyüklüğü 72 piksel olacak.
font-family: verdana; // font adı verdana olacak.
text-align: center; // metin ortalı yazılacak.
}
```

4.4. Birden çok elaman için stil uygulama

Aynı biçimlendirme özelliği birden çok elemana aynı anda uygulanabilir. Aşağıdaki örnek geri plan rengini sarı yapmakta olup bunu header, aside ve footer elemanlarının üçü için de yerine getirmektedir.

```
header, aside, footer {
background-color: yellow;
}
```

4.5. Stil ekleme yöntemleri

HTML ile çalışırken bir etiketin nerede görünmesini istiyorsanız oraya yerleştirir ve etkisini görürsünüz. Bununla birlikte stiller o kadar basit değildir. Bir web sayfasında stiller ile çalışmanın üç ana yolu vardır:

- Satır içi stiller (**Inline styles**) // modüler değil (adhoc) // satırın tamamında
- Dâhili stiller (**Internal styles**) // yarı modüler (semi adhoc / semi methodological) // sayfanın tamamında
- Harici stiller (**External styles**) // modüler (metodolojik) // sitenin tamamında

4.5.1. Satır içi stil ekleme

Satır içi stil, biçimlendirmek istediğiniz etikete doğrudan eklediğiniz bir stil kuralıdır. Sözdizimi aşağıdaki gibi verilebilir:

<element style="*property1*: *value1*; *property2*: *value2*; ...">

<h1 style="font-size: 72px;">Satır içi stil örneği görüyorsunuz.</h1>

* Bu başka bir satır içi stil örneği *

<h1>Stil uygulanmamış heading1 örneği</h1>

Her ikisi de h1 etiketi olmasına rağmen ilk satırdaki farklı diğeri farklı sonuç verecektir.

4.5.2. Dâhili stil ekleme

Satır içi stiller yararlı araçlardır, ancak etiketlerin içine sıkıştırıldıklarından ve sayfanın her yerine dağılmış olduklarından bakımları zordur.

Ayrıca, belirli bir stil kuralının birden fazla sayfa öğesine uygulanmasını isteme olasılığınız daha yüksektir. Bu durumda dâhili stil kullanımı daha uygundur.

Dâhili stiller <head> </head> blokları içerisinde yer alır ve genel olarak aşağıdaki gibi yazılırlar:

```
<!DOCTYPE html>
<head>
<style>
h1 {
border-width: 2px;
border-style: solid;
border-color: black;
}
h2 {
border-width: 2px;
border-style: dotted;
border-color: black;
}
</style>
</head>
```

Dâhili olarak yazılan stiller aynı sayfanın <body>…</body> bölümünde kullanılabilirler.

4.5.3. Harici stil ekleme

Stil kurallarını harici bir dosyada oluşturarak kullanma imkânı da bulunmaktadır.

Harici olarak bir stil dosyasına stilleri eklemek ve bunu kaydederek HTML dosyasından çağırmak metodolojik yaklaşım açısından daha uygun bir yöntemdir. Bir harici stil dosyası «.css» uzantısı (örneğin, «styles.css») ile kaydedilir. Dosyayı HTML dosyanızla aynı klasöre kaydedebilir veya bir alt klasör (örneğin, css veya stiller olarak adlandırılmış) oluşturabilirsiniz. Stilleri kullanmak istediğiniz her sayfa için sayfanın <head> bölümünün içinde bir <link>

etiketi eklenir. Kullanılacak genel biçim şu şekildedir (burada «filename.css», harici stil sayfası dosyasının adı olmak üzere):

```
     <link rel="stylesheet" href="filename.css">
```

Örnek bir harici stil dosyası aşağıdaki gibi olabilir:

```
h1 {
color: red;
}
p {
font-size: 16px;
}
```

Harici stil dosyası eklendikten sonra buradaki kurallara uygun şekilde çalışan h1 ve p etiketlerimizi kullanabiliriz. Yani h1 etiketi kırmızı renkli, p etiketi ise 16 piksel boyutunda etkiye sahip olacaktır. Mevcut etiket özellikleri korunacak olup bu özellikler de eklenmiş olacaktır.
Harici stil dosyasının uygulandığı örnek aşağıda görülmektedir:

```
<!DOCTYPE html>
<head>
     <link rel="stylesheet" href="filename.css">
</head>
<body>
<h1>Bu başlık kırmızı renkte görünecek </h1>
<p>Bu metin 16 piksel boyutunda görünecek </p>
</body>
</html>
```

4.6. CSS Seçicileri

Seçicilerin kullanımı ile her bir öge için biçimlendirme yapmak yerine sınıf ya da id için biçimlendirme yapılır ve bu biçimlendirme diğer ögelere uygulanır.
Sıklıkla kullanılan iki seçici CSS seçici ve ID seçicidir.

- CSS seçicisi web projelerinde tekrar tekrar ve farklı ögelere uygulanabilir.
- ID seçicisi, bir HTML belgesinde yalnızca bir öğeye uygulanabilir. Bir ID değeri bir sayfada yalnızca bir kez kullanılmalıdır.

4.6.1. Class seçiciler

Class seçiciler için örnek yazım aşağıdaki gibidir:

```
.class-name {
property1: value1;
property2: value2;
...
}
```

Kullanım:

```
<element class="class-name"> … </element>
```

Adı caption olan bir class için stil yazımı aşağıdaki gibi olacaktır:

```
.caption {
font-size: .75rem;
font-style: italic;
}
```

Kullanım:

```
<div class="caption"> … </div>
```

4.6.2. ID seçiciler

ID seçiciler için yazım şekli ve kullanımı aşağıda örneklenmiştir:

CSS Yazımı:

```
#id-name {
property1: value1;
property2: value2;
...
}
```

Kullanım:

```
<element id="id-name"> … </element>
```

Subtitle isimli bir ID seçici için yazım aşağıdaki şekildedir:

CSS Yazımı:

```
#subtitle {
font-size: 2rem;
font-style: italic;
color: blue;
}
```

Kullanım:

```
<div id="subtitle"> … </div>
```

Örnek 1: yerleşimi iki yana yaslı olan, rengi mavi olan, font büyüklüğü normalin %50 fazlası olan, italik bir stil özelliğini ornek1 adında bir class seçicisi olarak tanımla ve bunu p etiketiyle birlikte kullan.

Örnek 2: içeriden 20 piksel girintisi olan, alt çizgisi olan, ortalı, font adı verdana olan, boyutu 1.5 rem olan bir stili ID seçici yardımıyla oluştur ve bunu h3 etiketine uygula.

4.7. Metinler

Web sayfalarının en temel unsurlarından birisi metinlerdir.

4.7.1. Metin biçimlendirme stilleri

Metinlerle ilgili ilk olarak biçimlendirme stilleri uygulanacak olup aşağıdaki tabloda özetlenmiştir.

Özellik	Örnek	Açıklama
font-size	font-size: 16px;	Metnin boyutunu ayarlar.
font-family	font-family: serif;	Metnin yazı tipini ayarlar.
font-weight	font-weight: bold;	Metnin bold olup olmayacağını ayarlar.

font-style	font-style: italic;	Metnin italik olup olmayacağını ayarlar.
text-decoration	text-decoration: underline;	Metinde alt çizgi olup olmayacağını ayarlar.
text-align	text-align: justify;	Metni yatayda hizalar.
text-indent	text-indent: 8px;	İlk satırın ne kadar içeriden başlayacağını belirtir.

Örneğin bir paragrafta yer alan metinlerin tamamının boyutunun 20 piksel olmasını istiyorsak aşağıdakini yazabiliriz.

```
p {
font-size: 20px;
}
```

Kimi zaman bağlantılarda yer alan alt çizgi efektini kaldırmak isteriz. O zaman yapmamız gereken şey aşağıdaki gibi bir stil tanımlaması yapmaktır:

```
a {
text-decoration: none;
}
```

Eğer alt çizgi olmasını istersek text-decoration: underline; yazarız.
Bir metnin yönlendirmesi de ihtiyaç duyduğumuz özelliklerden biridir. Metin yönlendirmede aşağıdaki gibi stil tanımlaması yapabiliriz:

```
h1 {
text-align: right;
}
```

Yazılan stile göre h1 etiketi için yazının yönlendirmesi sağa dayalı olacaktır. Text-align için diğer özellik değerleri aşağıdaki gibi verilebilir:

```
    text-align: left | right | center |justify;
```

Etiketler içerisinde yer alan metinlerin girintili olarak görünmesini istediğimizde text-indent özelliğini kullanırız. Ne kadar girinti olacağını da değer olarak veririz. Aşağıdaki örnekte p, h1, h2 ve table etiketleri 12 piksel içeriden başlayacaktır.

```
p, h1, h2, table {
text-indent: 12px;
}
```

Yukarıda verilen tablodaki örneklere benzer şekilde çeşitli stiller oluşturmak mümkündür.
Örnek 1: satır içi stil yazma yöntemini kullanarak metin boyutu 16 piksel, metin rengi kırmızı, geri plan rengi sarı, font adı serif ve alt çizgisi olan bir stil oluşturunuz.
Örnek 2: dâhili stil yazma yöntemini kullanarak mavi renkli, 20 piksel boyutlu, ortalı bir metin stili yazınız.

4.7.2. Metin boyutlandırma stilleri

Metinlerle çalışırken boyutla ilgili özelliklere de ihtiyaç duyarız. Boyutlar konusunda karşımıza çıkan değerler aşağıdaki gibi verilebilir.

Birim	Ad	Tip	Eşittir
px	pixel	Absolute	Bir inch uzunluğun 96'da biri. (1 inch = 2,54 cm)
pt	point	Absolute	Bir inch uzunluğun 72'de biri.
em	em	Relative	Bir elemanın geçerli değeri, kalıt olarak aldığı ya da tanımlı font boyutu
rem	root em	Relative	Web sayfasının kök elemanının font büyüklüğü

vw	viewport width	Relative	Browser'ın içerik alanının geçerli genişliğinin 100'de biri
vh	viewport height	Relative	Browser'ın içerik alanının geçerli yüksekliğinin 100'de biri

Font-size ile temel boyutlandırma aşağıdaki gibi yapılabilir.

```
p { font-size: 16px; }          /* sabit piksel */
h1 { font-size: 32px; }
small { font-size: 12px; }
```

Göreli birimler

- **rem**: kök (html) font boyutuna göre (genelde en stabil)
- **em**: bulunduğu elemanın font boyutuna göre (iç içe büyüyebilir)

```
html { font-size: 16px; }

body { font-size: 1rem; }       /* 16px */
h2   { font-size: 1.5rem; }     /* 24px */
.note { font-size: 0.875rem; }  /* 14px */

.card { font-size: 16px; }
.card .title { font-size: 1.25em; } /* 20px (16px'e göre) */
```

4.7.2.1. Responsive boyut: clamp()

Ekran büyüdükçe metin büyür, ama min–max aralığında kalır.

```
h1 {
  font-size: clamp(24px, 3vw, 44px);
}

p {
  font-size: clamp(14px, 1.2vw, 18px);
}
```

4.7.2.2. Yüzde ile boyutlandırma

Genelde “kalıtım” üzerinden çalışır.

```
body { font-size: 16px; }
aside { font-size: 90%; }   /* 14.4px */
footer { font-size: 80%; }  /* 12.8px */
```

4.7.2.3. Satır aralığı: line-height

Satır aralığı vermek okunabilirlik için önemlidir. Aşağıda örnekler bulunmaktadır.

```
body { line-height: 1.6; }      /* önerilen aralık */
h1   { line-height: 1.2; }
p    { line-height: 1.7; }
```

4.7.2.4. Harf aralığı ve kelime aralığı

```
h1 { letter-spacing: -0.02em; } /* başlıklarda hafif sıkı */
p  { letter-spacing: 0.01em; }  /* uzun metinde çok abartma */
p  { word-spacing: 0.05em; }
```

4.7.2.5. Toplu örnek

```
:root { font-size: 16px; }

.h1 { font-size: clamp(28px, 3vw, 44px); line-height: 1.15; }
.h2 { font-size: clamp(22px, 2.2vw, 34px); line-height: 1.2;  }
.h3 { font-size: clamp(18px, 1.8vw, 26px); line-height: 1.25; }

.body { font-size: 1rem; line-height: 1.7; }
.small { font-size: 0.875rem; line-height: 1.6; }
.muted { font-size: 0.875rem; opacity: 0.8; }
```

4.8. Renkler

Tarayıcılar, sayfaları varsayılan stilleri kullanarak oluştururken, bağlantı metnini varsayılan ve tanıdık bir mavi olarak göstermek dışında renklerle pek bir şey yapmaz. Ancak CSS bazı güçlü renk araçları sunar.

4.8.1. Metin renklendirmede genel kullanım:

```
  color: value;
```

Özel olarak da aşağıdaki gibi kullanabiliriz.

```
color: tomato;
color: rgb(255, 99, 71);
color: #ff6347;
```

4.8.2. Geri plana renk verme

```
background-color: value;
```

Örnek olarak bir sayfanın tamamında geçerli olmak üzere hem yazı rengi hem de geri plan için aşağıdaki yazılabilir:

```
body {
color: rgb(255,255,255);
background-color: rgb(0,0,0);
}
```

4.9. Resimler

Resimler üzerinde CSS uygularken yapılabilecek temel işlemler aşağıda verilmiştir:

- Responsive (ekrana göre ölçeklenme)
- Oran bozulmadan küçülüp büyüme
- Köşeleri yuvarlatma, gölge verme
- Ortalamak veya kapsayıcıya sığdırmak

4.9.1. Temel özellikler:

Aşağıdaki örnek içinde bulunduğu alanın %100 genişliğinde ve ona uyumlu yüksekliğinde sunulur.

```
img {
  max-width: 100%;   /* Taşmayı engeller */
  height: auto;      /* Oranı korur */
}
```

4.9.2. Yuvarlak köşeler

Resimlerde yuvarlak köşeler yapmak istersek:

```
img {
  border-radius: 12px;
}
```

Tam daire şeklinde bir resim elde etmek için;

CSS bölümünde:

```
.profile {
  width: 150px;
  height: 150px;
  border-radius: 50%;
  object-fit: cover;
}
```

HTML bölümünde:

```
     <img src="profil.jpg" class="profile">
```

4.9.3. Gölge efekti

Resimlere gölge efekti vermek için;

```
img {
  box-shadow: 0 4px 10px rgba(0,0,0,0.25);
}
```

4.9.4. Resim ortalama

Resmi ortalamak için:

```
img {
  display: block;
  margin: auto;
}
```

4.9.5. Resim arkaplanı

Bir resmi arkaplan olarak kullanmak için:

```
.box {
  width: 300px;
  height: 200px;
  background-image: url("resim.jpg");
  background-size: cover;
  background-position: center;
}
```

4.10. Listeler

HTML'de sıralı, sırasız ve açıklama listeleri oluşturulabilir. Aşağıdaki örnekte basit olarak sırasız (madde imleri ile) liste oluşturmanın temel kodu verilmiştir:

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

4.10.1. Varsayılan işaretleri kaldırma

Sırasız listelerde yer alan madde imlerini için aşağıdaki stil yazılır;

```
ul {
  list-style: none;
  padding: 0;
```

```
}
```

4.10.2. Liste elemanları başına ikon yerleştirme

Her bir liste elemanının başına ikon gibi işaret koymak istersek;

```
li::before {
  content: "✓ ";
  color: green;
}
```

4.10.3. Yatay menü listesi

Normal şartlarda listeler dikey elemanlar olmasına rağmen CSS yardımıyla yatay listeler oluşturulabilir. Bunun için yapılması gereken liste elemanlarının yazılacağı alanı bir flexbox olarak tanımlamaktır.

```
ul {
  display: flex;
  gap: 20px;
}

li {
  list-style: none;
}
```

4.10.4. Liste elemanlarına hover efekti

Liste elemanlarına hover efekti eklemek için;

```
li {
  padding: 8px 12px;
  cursor: pointer;
}

li:hover {
  background: #f0f0f0;
  border-radius: 6px;
```

```
}
```

4.10.5. Numaralı liste

Numaralı liste oluşturmak için;

```
ol {
  list-style-type: upper-roman; /* I, II, III */
}
```

4.11. Tablolar

HTML'de tablo oluşturmak için aşağıdaki gibi kodlama yapılır.

```
<table>
  <tr>
    <th>Ad</th>
    <th>Bölüm</th>
  </tr>
  <tr>
    <td>Ali</td>
    <td>Bilgisayar</td>
  </tr>
</table>
```

4.11.1. Kenarlık birleştirme

Kenarlıkları birleştirmek için aşağıdakiler yazılır;

```
table {
  border-collapse: collapse;
}
```

4.11.2. Çerçeve özellikleri

Hücre çerçevesi için;

```
table, th, td {
 border: 1px solid #ccc;
```

```
}
```

4.11.3. Başlık satırı renklendirme

Başlık satırını renklendirme için;

```
th {
  background: #2f80ed;
  color: white;
}
```

4.11.4. Çizgili tablo

Zebra (çizgili) tablo elde edebilmek için;

```
tr:nth-child(even) {
  background: #f2f2f2;
}
```

4.11.5. Satır için hover

Tablo satırları için hover efekti aşağıdaki gibi verilebilir.

```
tr:hover {
  background: #ddd;
}
```

4.11.6. Responsive tablo

Responsive – kaydırmalı tablo için;

```
.table-container {
  overflow-x: auto;
}
```

HTML bölümünde ise aşağıdakiler yazılır;

```
<div class="table-container">
  <table>...</table>
</div>
```

4.11.7. Toplu örnek

Tablolarla ilgili birçok özellik bir arada aşağıdaki örnekte verilmiştir.

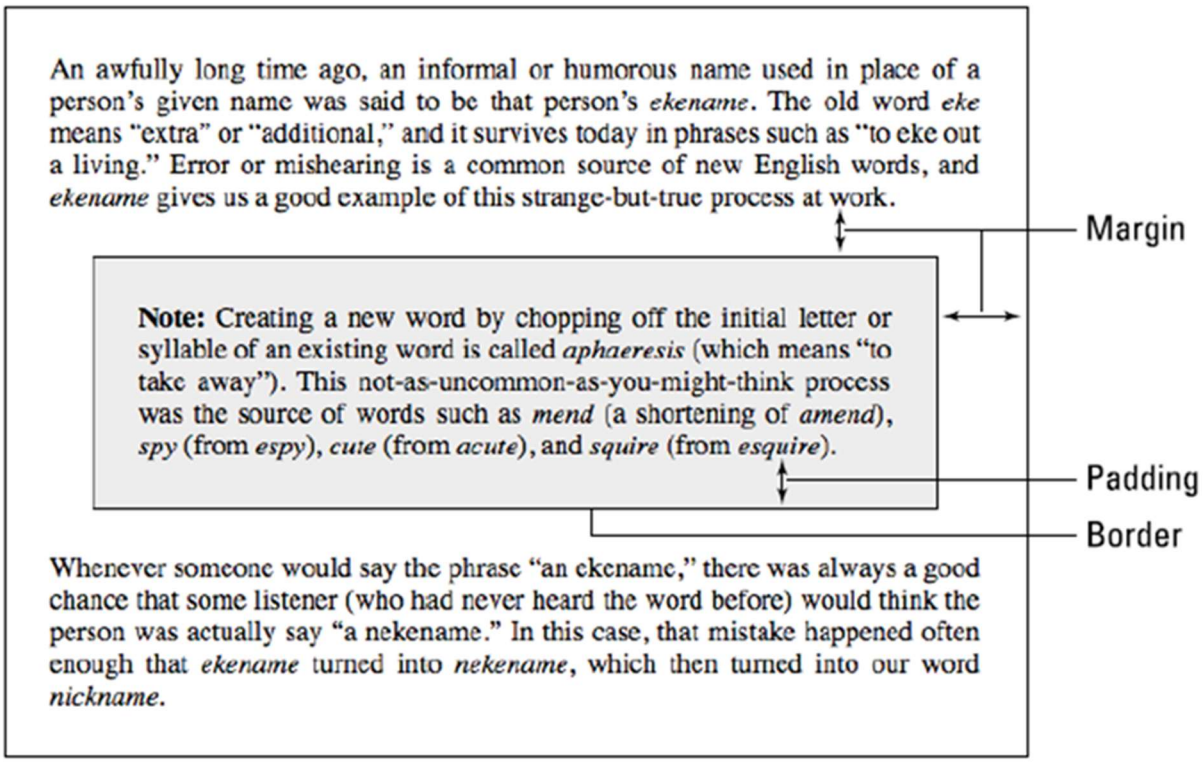
```
<img src="foto.jpg" class="profile">

<ul class="menu">
  <li>Anasayfa</li>
  <li>Hakkımızda</li>
  <li>İletişim</li>
</ul>

<table class="data">
  <tr>
    <th>Ders</th>
    <th>Kredi</th>
  </tr>
  <tr>
    <td>Veri Madenciliği</td>
    <td>3</td>
  </tr>
</table>
.profile {
  width: 120px;
  border-radius: 50%;
  display: block;
  margin: auto;
}

.menu {
  display: flex;
```

```
  gap: 15px;
  list-style: none;
}

.data {
  border-collapse: collapse;
  width: 100%;
}

.data th, .data td {
  border: 1px solid #ccc;
  padding: 8px;
}

.data th {
  background: #444;
  color: #fff;
}
```

4.12. CSS kutu (box) modeli

Her web sayfası bir dizi HTML etiketinden oluşur ve bu etiketlerin her biri sayfadaki bir öğeyi temsil eder. Web sayfalarında ögelerin her birinin etrafında görünmez bir kutu olduğu kabul edilir.

Her bir etiketin, hatta <a> ve gibi satır içi öğelerin bile etraflarında bir kutu vardır. CSS Box Model web sayfa tasarımlarındaki görünmez kutuları oluşturmayı konu alan önemli bir modeldir.



Şekil 1. CSS Box Model Örneği

- **Content (içerik)**: Kutunun içindekileri (metin, resimler vb.) tanımlar.
- **Padding (dolgu)**: İçerik (content) ile o içeriği sınırlayan çizgi (border) arasındaki boşluğu tanımlar.
- **Border (kenarlık)**: Kutu dolgusunu çevreleyen çizgiyi tanımlar.
- **Margin (kenar dışındaki boşluk)**: Kutuyu diğer kutulardan soldan, sağdan, üstten ve alttan ayıran kenarlıkların dışındaki boşluğu tanımlar.
- **Dimensions (boyutlar)**: Kutunun sayfa içindeki konumunu ve boyutlarını tanımlar.

4.12.1. Padding (dolgu)

Dolgunun (padding) dört bölümü vardır:

padding-top: içeriğin üstünde kalan boşluk.

padding-right: içeriğin sağında kalan boşluk.

padding-bottom: içeriğin altında kalan boşluk.

padding-left: içeriğin solunda kalan boşluk.

```
element {
     padding-top: top-değeri;
     padding-right: right-değeri;
     padding-bottom: bottom-değeri;
```

```
    padding-left: left-değeri;
    }
```

4.12.1.1. Padding için değer atama

Yazım	Açıklama
padding: değer1;	Bütün kenarlara değer1 uygulanır.
padding: değer1 değer2;	top (üst) ve bottom (alt) için değer1, right (sağ) ve left (sol) için değer2 uygulanır.
padding: değer1 değer2 değer3;	top (üst) için değer1, right (sağ) ve left (sol) için değer2, bottom (alt) için değer3 uygulanır.
padding: değer1 değer2 değer3 değer4;	top (üst) için değer1, right (sağ) için değer2, bottom (alt) için değer3 ve left (sol) için değer4 uygulanır.

Kısa yoldan atama	.padding-ekleme {
	padding: 1rem 1.5rem 0.5rem 1.25rem;
	}

4.12.1.2. Padding için örnek 1

sample-padding isimli bir class tanımlayarak yukarıdan 1rem (root em - kök elemanın font boyutu - aksi belirtilmedikçe 13px), sağdan 1.5 rem, alttan 0.5 rem ve soldan 1.25 rem olacak şekilde boşluk tanımlayın ve bunu h3 etiketiyle birlikte kullanın.

Class yazımı (<head>..</head> içinde <style>..</style> arasında ya da harici bir css dosyasında)

```
.sample-padding {
        padding-top: 1rem;
        padding-right: 1.5rem;
```

```
        padding-bottom: 0.5rem;
        padding-left: 1.25rem;
}
```

Kullanım (<body>..</body> içinde)

```
<h3 class="sample-padding"> Örnek padding </h3>
```

4.12.1.3. Padding için örnek 2

Yukarıdaki örnek için (içeride yer alan Note ile başlayan metin bölümü) adı kutu olan bir class yazılmış ve metne uygulanmıştır.

Class yazımı:

```
    .kutu {
        padding-top: 20px;
        padding-right: 20px;
        padding-bottom: 20px;
        padding-left: 20px;
    }
```

<u>Kullanım:</u>

```
<div class="kutu">
<b>Note:</b> Creating a new word by chopping off the initial letter or syllable of an existing word is called aphaeresis (which means "to take away"). This not-as-uncommon-as-you-might-think process was the source of words such as mend (a shortening of amend), spy (from espy), cute (from acute), and squire (from esquire).
</div>
```

4.12.2. Border (kenar)

Bir öğenin sınırı, öğenin içeriğini ve dolgusunu çevreleyen çizgi kümesidir. Kenarlıklar, bir öğenin kendisini çevreleyen öğelerden ayrı olduğunu netleştirmenin yararlı bir yoludur.

Bir öğenin kenarlığıyla ilişkilendirilmiş dört çizgisi vardır

- **border-top**: dolgunun üstündeki kenarlığı tanımlar,

- **border-right**: dolgunun sağındaki kenarlığı tanımlar,
- **border-bottom**: dolgunun altındaki kenarlığı tanımlar,
- **border-left**: dolgunun solundaki kenarlığı tanımlar.

Her bir kenarlığın da kendi içinde özellikleri vardır:

```
element {
border-top: top-width top-style top-color;
border-right: right-width right -style right -color;
border-bottom: bottom-width bottom-style bottom-color;
border-left: left-width left-style left-color;
}
```

Yukarıda görüleceği üzere her bir border elemanı üç farklı değerden oluşur: **width-style-color** (genişlik-stil-renk). Aşağıda bir örnek verilmiştir.

```
     header { border-bottom: 1px dashed red; }
```

Örneğe göre header elemanının alt kenarlığı 1px genişliğinde, kesikli stilde ve kırmızı renkte olacaktır. Eğer sadece bottom elemanı değil de bütün kenarlar için değerleri aynı anda vermek istersek:

```
     header { border: 1px solid black; }
```

Yazabiliriz. Bu yazımın çıktısı bütün kenarların genişliği 1px, stili solid (kesintisiz) ve siyah renkli olacaktır.

4.12.2.1. Border için örnek 1

Bir önceki padding örneği border ekleyerek güncellenmiştir. Buna göre içerideki metni verecek kutu isimli stil aşağıdaki hale gelmiştir.

```
     .kutu {
          padding-top: 20px;
          padding-right: 20px;
          padding-bottom: 20px;
```

```
        padding-left: 20px;
        border: 1px solid black;
    }
```

Bu kutunun uygulanması bir önceki örnekle aynıdır. Şekil 1'de bir iç kutu bir de dış kutu olduğu için dış kutuya özel ek bir stil yazılmalıdır. Bu stil de şu şekilde yazılabilir.

```
    .diskutu {
        padding-top: 30px;
        padding-right: 30px;
        padding-bottom: 30px;
        padding-left: 30px;
        border: 1px solid black;
    }
```

Dışarıdaki metin ve içerideki metini iç içe ve padding bilgileriyle ekleyebilmek için aşağıdaki gibi yazım yapılmıştır.

```
<div class="diskutu">
    An awfully long time ago, an informal or humorous name used
in place of a person's given name was said to be that person's
ekename. The old word eke means "extra" or "additional," and it
survives today in phrases such as "to eke out a living." Error
or mishearing is a common source of new English words, and
ekename gives us a good example of this strange-but-true process
at work.
    <div class="kutu">
        <b>Note:</b> Creating a new word by chopping off the
initial letter or syllable of an existing word is called
aphaeresis (which means "to take away"). This not-as-uncommon-
as-you-might-think process was the source of words such as mend
(a shortening of amend), spy (from espy), cute (from acute), and
squire (from esquire).
    </div>
```

```
Whenever someone would say the phrase "an ekename," there was
always a good chance that some listener (who had never heard the
word before) would think the person was actually saying "a
nekename." In this case, that mistake happened often enough that
ekename turned into nekename, which then turned into our word
nickname.
</div>
```

İç içe iki div bölümü yazılmış, dışarıdaki div için diskutu stili kullanılmış içerdeki için ise kutu stili kullanılmıştır. Elde edilen çıktı şu şekildedir:

An awfully long time ago, an informal or humorous name used in place of a person's given name was said to be that person's ekename. The old word eke means "extra" or "additional," and it survives today in phrases such as "to eke out a living." Error or mishearing is a common source of new English words, and ekename gives us a good example of this strange-but-true process at work.

Note: Creating a new word by chopping off the initial letter or syllable of an existing word is called aphaeresis (which means "to take away"). This not-as-uncommon-as-you-might-think process was the source of words such as mend (a shortening of amend), spy (from espy), cute (from acute), and squire (from esquire).

Whenever someone would say the phrase "an ekename," there was always a good chance that some listener (who had never heard the word before) would think the person was actually saying "a nekename." In this case, that mistake happened often enough that ekename turned into nekename, which then turned into our word nickname.

Henüz margin kullanılmadığı için dışarıdaki metin ile içerdeki kutu arasında henüz bir ayrım yoktur. Ayrıca metin iki yana yaslı olmadığı için düzenli değildir. Metni düzeltmek için aşağıdaki satır stiller eklenmiştir:

```
    text-align: justify;
```

4.12.3. Margin

CSS kutu modelinin son bileşeni, kutunun kenarlığı etrafındaki boşluk olan kenar boşluğudur. Kenar boşlukları, öğelerin tarayıcı içerik alanının kenarları ile bütünleşik olmasını engeller ve öğeler arasında ayrım oluşturduğu için web tasarımında önemli bir ayrıntıdır.

Dolguda olduğu gibi, kenar boşluğunda kenarlığın üstünde, sağında, altında ve solunda olmak üzere dört bölüm vardır, bu nedenle CSS, bir öğeye kenar boşlukları eklemek için karşılık gelen dört özellik sunar:

```
element {
margin-top: kenarlık üstündeki boşluk;
margin-right: kenarlık sağındaki boşluk;
margin-bottom: kenarlık altındaki boşluk;
margin-left: kenarlık solundaki boşluk;
}
```

4.12.3.1. Margin parametreleri

Sentaks	Açıklama
`margin: değer1;`	Bütün kenarlara aynı değer uygulanır.
`margin: değer1 değer2;`	değer1 top ve bottom için değer2 right ve left için kullanılır.
`margin: değer1 değer2 değer3;`	değer1 top değerine, değer2 right ve left değerine, değer3 bottom değerine uygulanır.
`margin: değer1 değer2 değer3 değer4;`	değer1 top değerine, değer2 right değerine, değer3 bottom değerine ve değer4 left değerine uygulanır.

4.12.3.2. Margin için örnek 1

Şekil 1'deki görünümü elde etmek için margin özelliği de kullanılmıştır. Margin özelliği kenarlık etrafında boşluk oluşturduğu için içerikdeki kutuya uygulanmış dışarıdakine uygulanmamıştır. Kutu isimli stil için en güncel durum aşağıdaki gibidir:

```
    .kutu {
        padding-top: 20px;
        padding-right: 20px;
        padding-bottom: 20px;
        padding-left: 20px;
        border: 1px solid black;
        text-align: justify;
```

```
        margin-top: 20px;
        margin-right: 20px;
        margin-bottom: 20px;
        margin-left: 20px;
    }
```

Kutu isimli stil güncellendikten sonra elde edilen çıktı aşağıdaki gibi olmuştur:

An awfully long time ago, an informal or humorous name used in place of a person's given name was said to be that person's ekename. The old word eke means "extra" or "additional," and it survives today in phrases such as "to eke out a living." Error or mishearing is a common source of new English words, and ekename gives us a good example of this strange-but-true process at work.

Note: Creating a new word by chopping off the initial letter or syllable of an existing word is called aphaeresis (which means "to take away"). This not-as-uncommon-as-you-might-think process was the source of words such as mend (a shortening of amend), spy (from espy), cute (from acute), and squire (from esquire).

Whenever someone would say the phrase "an ekename," there was always a good chance that some listener (who had never heard the word before) would think the person was actually saying "a nekename." In this case, that mistake happened often enough that ekename turned into nekename, which then turned into our word nickname.

Böylece istenen çıktı elde edilmiştir.

4.12.4. Dimemsions

CSS kutu modelindeki son özellik ise boyut özelliğidir. Aksi belirtilmedikçe ekran boyutlarında çalışan stiller yerine hangi uzunluk ve genişlik istersek ona uygun değerler veririz. Genel kullanımı aşağıdaki gibidir:

```
width: value;
height: value;
```

Yukarıda diskutu olarak belirttiğimiz stil için ekranın yarısı boyutunda bir özellik tanımlamak istersek stil aşağıdaki gibi güncellenir.

```
        .diskutu {
        padding-top: 30px;
        padding-right: 30px;
        padding-bottom: 30px;
```

```
        padding-left: 30px;
        border: 1px solid black;
        text-align: justify;
        width: 50vw;
        height: 50vh;
    }
```

Stil yazımını kısaltmak istersek de aşağıdaki gibi yazabiliriz.

```
        .diskutu {
        padding: 30px;
        border: 1px solid black;
        text-align: justify;
        width: 50vw;
        height: 50vh;
    }
```

4.13. Floating elements (yüzen elemanlar)

Bir sayfadaki içeriği konumlandırmak ve biçimlendirmek için kullanılan elemanlara yüzen elemanlar adı verilir. Örneğin, sayfanın solunda yer alacak ve 25rem boyutunda olacak section elemanını aşağıdaki gibi tanımlayabiliriz:

```
section {
float: left;
width: 25rem;
}
```

Ya da sayfanın sağında ve 15 rem genişliğinde aside isimli bir elamanı şöyle tanımlayabiliriz:

```
aside {
float: right;
width: 15rem;
}
```

Yüzen elemanlar sıklıkla bir görsel ile bir metnin ilişkisini tanımlamakta kullanılır. Ekranın sol tarafında resim sağ tarafında metin olmasını istiyorsak <head> … </head> satırları arasında aşağıdaki stili ve <body> … </body> satırları arasında ise aşağıdaki kodu yazabiliriz.

```
<!DOCTYPE html>
<html>
    <head>
        <title> Floating elements </title>
          <style>
          img {
          float: left;
          height: 200px;
          margin-right: 10px;
          }
          </style>
     </head>
     <body>
     <img src="resim1.jpg">
     Bir sayfadaki içeriği konumlandırmak ve biçimlendirmek için
     kullanılan elemanlara yüzen elemanlar adı verilir
     </body>
</html>
```

Çıktı:

Bir sayfadaki içeriği konumlandırmak ve biçimlendirmek için kullanılan elemanlara yüzen elemanlar adı verilir

Resmin solda değil de sağda olmasını istersek o zaman stil kısmını aşağıdaki gibi güncelleriz.

```
<style>
img {
float: right;
height: 200px;
margin-right: 10px;
}
</style>
```

Floating elements yatayda harekete izin veriyor.

4.14. Positioning elements (konumlanan elemanlar)

Yüzen elemanlar gibi konumlanan elemanlar da bir sayfadaki içeriği konumlandırmak için kullanılır. Bununla birlikte yüzen elemanlar içeriklerin sol, sağ ya da ortada olması gibi kısıtlı bir imkânı desteklerden konumlanan elemanlar daha fazlasını sağlar. Konumlanan elemanlar sayesinde içeriğin hem yatay da hem de dikeyde konumlandırması yapılabilir. Ayrıca sabit kalan içerikler ya da scrool ile birlikte hareket eden içerikler elde edebiliriz.
Genel kullanımı şöyledir:

```
element {
     position: static | relative | absolute | fixed;
}
```

4.14.1. Position: static

Normal bir HTML etiketi nasıl çalışıyorsa öyle çalışır. Tüm HTML elementlerinin varsayılan konumudur. top, left, right, bottom çalışmaz. Normal belge akışına göre yerleşir.

```
div {
  position: static;
}
```

“Hiçbir şey yapmayan” position türü.

4.14.2. Position: relative

Element kendi normal konumuna göre kaydırılır. Sayfadaki yerini “korur”. top, left çalışır.

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}
```

Element yerinden oynar ama boşluğu korunur. “Ben buradayım ama biraz kayayım.”

4.14.3. Position: absolute

Kendisini ana çerçeveye göre sabit olarak konumlandırır. Normal akıştan çıkar. En yakın position: relative|absolute|fixed|sticky olan üst elemana göre konumlanır. Eğer böyle bir üst element yoksa → body’ye göre konumlanır.

```
.container {
  position: relative;
}

.box {
  position: absolute;
  top: 10px;
  right: 20px;
}
```

Element akıştan tamamen kopar.

4.14.4. Position: fixed

Tarayıcı ekranına göre konumlanır. Scroll yapılsa bile yerinden oynamaz.

```
.menu {
  position: fixed;
  top: 0;
```

```
  left: 0;
}
```

Diğerlerinden en önemli farkı ekrana sabitlenir. Sayfa kaysa da fixed pozisyondaki sabit kalır. Tipik olarak; Navbar, WhatsApp butonu veya Chat butonu için kullanılır.

4.14.5. Position: sticky

Normalde relative gibi davranır. Scroll sırasında belirli noktada fixed olur.

```
.header {
  position: sticky;
  top: 0;
}
```

Diğerleriyle arasındaki fark scroll ile davranışının değişmesidir. Başta normal olan pozisyon sonra yapışır. Tipik olarak; Başlıklar, Tab menüler ve Filtre çubukları için kullanılır.

4.15. CSS Flexbox

Flexbox (Flexible Box Layout), web sayfalarında **elemanları hizalamayı, sıralamayı ve boşlukları yönetmeyi** kolaylaştıran modern bir CSS düzen (layout) sistemidir. Amacı "Bir container içindeki elemanları akıllı ve esnek şekilde yerleştirmek" olarak özetlenebilir. Flexbox iki parçadan oluşur:

Ebeveyn (flex container): flex elemanları tutacak çatı elemandır.

```
.container {
  display: flex;
}
```

Flex çocuklar (flex items): flex içinde kalacak elemanlardır.

```
<div class="container">
  <div>1</div>
  <div>2</div>
```

```
  <div>3</div>
</div>
```

Flexbox'ta hizalama **iki eksene göre** yapılır:

Main Axis → (justify-content)

Cross Axis ↓ (align-items)

flex-direction (Yön)

```
.container {
  display: flex;
  flex-direction: row; /* varsayılan */
}
```

Değer	Açıklama
row	Soldan sağa
column	Yukarıdan aşağı
row-reverse	Ters yatay
column-reverse	Ters dikey

justify-content (Ana eksende hizalama)

```
.container {
  display: flex;
  justify-content: center;
}
```

Değer	Etki
flex-start	Başta
center	Ortada
flex-end	Sonda

Değer	Etki
space-between	Kenarlara yasla
space-around	Etraflı boşluk
space-evenly	Eşit boşluk

align-items (Çapraz eksende hizalama)

```
.container {
  display: flex;
  align-items: center;
}
```

Değer	Açıklama
stretch	Varsayılan (uzar)
center	Dikey ortala
flex-start	Üste
flex-end	Alta

Ortalanmış kutu:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

gap (Elemanlar arası boşluk)

```
.container {
  display: flex;
  gap: 20px;
```

```
}
```

flex-grow (Büyüme oranı) 1 değerini aldığında flex itemler bütün ekran genişliğini kapatacak şekilde büyür.

```
.box {
  flex-grow: 1;
}
```

Küçülme oranı

```
.box {
  flex-shrink: 0;
}
```

Değerin 0 olması küçülmeyi engeller.

flex-basis (Başlangıç boyutu)

```
.box {
  flex-basis: 200px;
}
```

4.16. Grid

CSS Grid, web sayfalarında **iki boyutlu (satır + sütun)** yerleşimler oluşturmak için kullanılan modern bir layout sistemidir. Flexbox tek boyutlu (row *veya* column) iken, Grid hem yatay hem dikey düzen sağlar.

Grid container: grid için ebeveyn

Grid item: container içindeki elemanlar

Row / column: satır ve sütunlar

Gap: hücreler arası boşluk

Aşağıda tanımlanan container sınıfı bir ebeveyn grid tanımlar:

```
.container {
display: grid;
}
```

Ebeveyn grid içerisinde ilk kolonu 200 piksel, ikinci ve üçüncü kolonu oransal boşluk içeren üç kolonlu ve ilk satırı 100 piksel diğeri içeriğe göre değişen boyutta bir satır içeren bir yapı olacak şekilde tanımlama aşağıda görüldüğü gibi olacaktır.

```
.container {
display: grid;
grid-template-columns: 200px 1fr 1fr;
grid-template-rows: 100px auto;
}
```

Eğer üçü de eşit boyutlu kolonlar oluşturmak isteseydik;

```
grid-template-columns: repeat(3, 1fr);
```

Bir grid içerisinde satır ve sütunlar arasında 20 piksel boşluk olmasını istersek;

```
gap: 20px;
```

Satırlar arasında 10 piksel ve sütunlar arasında 30 piksel mesafe için aşağıdakileri yazabiliriz.

```
row-gap: 10px;
column-gap: 30px;
```

Ekran küçüldükçe kolon sayısı otomatik azalması için aşağıdaki yazılır.

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Ögeleri konumlandırma için aşağıdaki gibi bir yazım tercih edilebilir.

```
.item1 {
grid-column: 1 / 3;   /* 1'den başla 3'e kadar */
```

```
grid-row: 1 / 2;
}
```

Grid yapısıyla birlikte semantik alanları yönetmek için aşağıdaki class kullanılabilir.

```
.container {
grid-template-areas:
"header header"
"menu content"
"footer footer";
}

.header { grid-area: header; }
.menu { grid-area: menu; }
.content { grid-area: content; }
.footer { grid-area: footer; }
```

Bu kullanım okunabilirliği artırır.

Hizalama için;

Tüm grid için:

```
justify-items: center;   /* yatay */
align-items: center;     /* dikey */
```

Tek eleman için:

```
.item {
justify-self: end;
align-self: start;
}
```

Mini bir örnek:

```
<div class="grid">
<div>A</div>
<div>B</div>
<div>C</div>
<div>D</div>
</div>

.grid {
display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 10px;
}
```

5. JAVASCRIPT

5.1. Javascript Giriş

JavaScript HTML sayfalarına davranış ve etkileşim kazandırmak için kullanılan bir script dilidir. 1995 yılında Netscape firması tarafından geliştirilmiştir (LiveScript). Ayrıca son dönemde türlere uygun sözdizimini de destekleyen TypeScript sürümü geliştirilmiştir. TypeScript, JavaScript üzerine kurulu, güçlü tip denetimine sahip bir programlama dilidir. JavaScript ile Java arasında bir ilişki yoktur.

Javascript kodları kullanıcının web tarayıcısında (sunucu değil) yerleşik bir yorumlayıcı tarafından çalıştırılır.

Script dosyaları HEAD veya BODY bölümü içerisinde bir yere konabilir. HEAD bölümünde ve yerel olarak script dosyaları bulunduğunda komut dosyaları sayfa görüntülenmeden önce çalışır. BODY bölümünde bulunduğunda ise sayfa görüntülenirken komut dosyaları yüklenir ve çalışır.

Basit bir örnek aşağıda verilmiştir:

```
<html>
<head>
     <title>Hello World</title>
</head>
<body>
<script type="text/javascript">
document.write("Hello World!");
</script>
</body>
</html>
```

Ekranda Hello World ifadesini yazdıran bu kullanım **doğrudan kullanım** olarak bilinir. Alternative olarak aşağıdaki gibi çalıştırma yapılabilir.

```
<html>
```

```
<head>
<script type="text/javascript">
function helloWorld() {
document.write("Hello World!");
}
</script>
</head>
<body>
<script type="text/javascript">
helloWorld();
</script>
</body>
</html>
```

Bu kullanım ise dâhili kullanım olarak adlandırılır. Hem doğrudan veya satıriçi kullanım hem de dâhili kullanım geçici çözüm yöntemidir. Metodolojik çözüm ise script komutlarının harici bir dosya olarak yazılması ve dosya olarak eklenmesidir. Harici bir javascript dosyası aşağıdaki gibi eklenebilir.

```
      <script src="app.js"></script>
```

Bu kullanımın bir alternatifi de script dosyasının HTML yüklendikten sonra çalışması için defer kullanımıdır. Bahsedilen kullanım aşağıdaki gibidir:

```
     <script src="app.js" defer></script>
```

5.1.1. Javascript değişkenleri

Diğer dillerde olduğu gibi JavaScript de değişkenlere sahiptir. Değişken tanımı için klasik olarak var anahtar sözcüğü kullanılır. Fakat şu anda değişken tanımlama için let yoğun olarak kullanılır.

```
     let count = 0;
```

Ayrıca sabit değerler için de const ifadesi kullanılır.

```
    const pi = 3.14;
```

Örnek: İsim ve yas değişkenlerini tanımlayarak değerleri ekranda gösteren kodu yazınız.

```
<!doctype html>
<html>
<body>
<script type="text/javascript">
let isim = "Hidayet";
const yas = 45;
console.log("İsim:", isim);
console.log("Yaş:", yas);
</script>
</body>
</html>
```

Yukardaki kod parçası tarayıcının konsolunda çıktı üretir. Çıktıyı görebilmek için F12 -> console yolunu takip etmeniz gerekir.

5.1.2. Veri tipleri

JavaScript veri tipleri aşağıdaki türlerdedir.

```
string, number, boolean
null, undefined
object, array
const user = { name: "Ali", age: 25 };
const list = [1, 2, 3];
```

Veri tipleri içinde önemli yapılardan biri de dizilerdir. Dizilerle ilgili aşağıda bir örnek verilmiştir.

```
let colors = new Array();
colors[0] = "red";
colors[1] = "green";
```

```
colors[2] = "blue";
colors[3] = "orange";
colors[4] = "magenta";
colors[5] = "cyan";
for (let i in colors) {
     document.write(colors[i]+"<br>");
}
```

Yukarıdaki örneğin for kısmı forEach yardımıyla aşağıdaki gibi de yazılabilir.

```
colors.forEach(function(renk) {
     document.write(renk+"<br>");})
```

5.1.3. Operatörler ve yapılar

JavaScript, C/Java'dan alışık olduğumuz operatörlerin çoğuna sahiptir.

- Aritmetik (Arithmetic) (+, - , *, /, %)
- Atama (Assignment) (=, +=, -=, *= /=, %=, ++, --)
- Mantıksal (Logical) (&&, ||, !)
- Karşılaştırma (Comparison) (<, >, <=, >=, ==, (!=,<>))

Not: + ayrıca dize birleştirme yapar.

- Yapılar: if, else, while, for, switch, case

5.1.4. Fonksiyonlar

JavaScript, function anahtar kelimesini kullanarak fonksiyonları tanımlamanıza olanak tanır. Fonksiyonlar, dönüş anahtar sözcüğünü kullanarak değer döndürebilir.

Aşağıdaki örnek ekranda bir onay penceresi çıkarmakta ve kullanıcıdan OK veya Cancel butonu beklemektedir.

```
function showConfirm() {
confirm("Yapmak istediğinizden emin misiniz?");
}
```

Bir başka örnek a ve b gibi iki sayının toplamını veren aşağıdaki gibi bir fonksiyondur.

```
function add(a, b) {
```

```
  return a + b;
}
```

Toplama gibi basit işlemler için fonksiyona alternatif olarak ise (=>) yapısı da kullanılabilir.

```
      const add = (a, b) => a + b;
```

Bir butona bağlı olarak bir fonksiyon çalıştırmak istersek aşağıdaki kodu yazabiliriz.

```
<button onclick="mesajGoster()">Tıkla</button>
<script>
function mesajGoster() {
  alert("Merhaba JavaScript!");
}
</script>
```

Yukarıdaki kod parçası aynı zamanda JavaScript olayları kapsamında da örnek teşkil eder.

5.1.5. Koşullar ve döngüler

En temel koşul ifadesi diğer dillerde olduğu gibi if ile ifade edilir:

```
     if (age >= 18) { ... }
```

Aşağıda bir koşul örneği verilmiştir.

```
let not = 75;

if(not >= 50){
   console.log("Geçti");
}else{
   console.log("Kaldı");
}
```

En genel döngü ifadesi olarak da for kullanılır:

```
    for (let i = 0; i < 5; i++) { ... }
```

Aşağıda döngülerle ilgili bir örnek verilmiştir.

```
for(let i=1; i<=5; i++){
  console.log(i);
}
```

forEach bir başka döngü ifadesidir:

```
    list.forEach(x => console.log(x));
```

5.1.6. Nesneler

JavaScript ile nesne tanımlamak ve kullanmak da mümkündür.

```
let ogrenci = {
  ad: "Ali",
  no: 123,
  bolum: "Bilgisayar Mühendisliği"
};

console.log(ogrenci.ad);
```

Ayrıca JavaScript tarafından kullanılan yerleşik nesneler de bulunmaktadır.

En bilinen JavaScript nesnesi document nesnesidir. HTML dokümanını yönetmek için kullanılır.

```
      document.write("Hello World!");
```

Diğer nesneler aşağıdaki gibidir:

```
String
```

Date
Array
Boolean
Math

Oluşturulan bir string her seferinde bir string sabiti ile birlikte kullanılır (Java gibi)

Java'da olduğu gibi birçok aynı metot bulunur:

- `charAt, concat, indexOf, lastIndexOf, match, replace, search, slice, split, substr, substring, toLowerCase, toUpperCase, valueOf`

Aynı zamanda HTML özel metotlar da bulunur

- `big, blink, bold, fixed, fontcolor, fontsize, italics, link, small, strike, sub, sup`

Date sınıfı:

- Date sınıfı, tarihlerle çalışmayı kolaylaştırır
- Mevcut tarih ile yeni bir tarih başlatılır.
- Tarihler karşılaştırılabilir ve artırılabilir

5.2. Document Object Model (DOM)

Doküman Nesne Modeli (Document Object Model - DOM), programlama dili nesneleri ile bir HTML belgesinin öğeleri arasındaki eşlemeyi belirleyen bir belirtimdir.

5.2.1. DOM nesneleri

Çevresel (environment) nesneler

- `Window, Navigator, Screen, History, Location, Document`

HTML nesneleri

- `Anchor, Area, Base, Body, Button, Event, Form, Frame, Frameset, Iframe, Image, Checkbox, FileUpload, Hidden, Password, Radio, Reset, Submit, Text, Link, Meta, Object, Option, Select, Style, Table, TableCell, TableRow, TextArea`

5.2.2. HTML DOM: Document

Doküman (Document) nesnesi bir HTML dokümanı sunar ve bir sayfadaki bütün dokümanlara erişim için kullanılabilir. Bir doküman birçok koleksiyon içerir:

- `anchors, forms, images, links`

Ayrıca birkaç özelliğe sahiptir:

- `body, cookie, domain, lastModified, referrer, title, URL`

Ek olarak birkaç faydalı metoda sahiptir

- **`getElementById`**, **`getElementsByName`**, `getElementsByTagName, write, writeln, open, close`

Document nesnesi ile ilgili bir örnek aşağıda verilmiştir.

```
<input type="text" id="ad" placeholder="Adınızı giriniz">
<button onclick="yazdir()">Gönder</button>
<p id="sonuc"></p>

<script>
function yazdir() {
  let ad = document.getElementById("ad").value;
  document.getElementById("sonuc").innerText = "Merhaba " +
ad;
}
</script>
```

Kod açıklama:

Ekranda id değeri “ad” olan bir textbox ve içerisinde “Adınızı giriniz” mesajı ile yanı başında “Gönder” başlığı yer alan bir buton bulunur. Buton tıklanınca (onclick olayı) yazdır() fonksiyonu çağrılır. yazdir() fonksionu DOM modeliyle çalışan bir fonksiyondur. Fonksiyon ilk olarak id değeri “ad” olan textbox içerisindeki değeri ad değişkenine alır. Ardından “sonuç” isimli id değeri ile verilen paragraf içerisinde önce “Merhaba” yazar ve ardından ad değişkeninin değerini yazdırır. Burada text alanından değer alırken **value** özelliği ve paragraf

içine değer yerleştirmek için **innerText** özelliği kullanılır. document.getElementById ise bizi ilgili alanlara ulaştıran metottur.

DOM nesnesi yardımıyla bir metnin stilini aşağıdaki gibi değiştirebilirsiniz.

```
<p id="metin">Merhaba Dünya</p>
<button onclick="renkDegistir()">Renk Değiştir</button>

<script>
function renkDegistir(){
      document.getElementById("metin").style.color = "blue";
}
</script>
```

5.3. Javascript olayları (events)

JavaScript olayları; kullanıcının web sayfasıyla etkileşime girdiğinde (tıklama, klavye kullanımı, fare hareketi, form gönderme vb.) tarayıcı tarafından tetiklenen durumlardır.

Olay	Açıklama
click	Tıklama olayı
dblclick	Çift tıklama olayı
mouseover	Nesnenin üzerine gelince
mouseout	Nesneyi terk edince
keydown	Tuşa basılınca
keyup	Tuş bırakılınca
submit	Form gönderilince
load	Sayfa yüklenince
change	Değer değişince

5.3.1. Click olayı ile ilgili örnekler

HTML içinde olay kontrolü aşağıdaki gibi yapılır.

```
<button onclick="mesajVer()">Tıkla</button>

<script>
function mesajVer(){
  alert("Butona tıklandı!");
}
</script>
```

Klasik olarak butona tıklanınca onclick mesajVer() fonksiyonunu çalıştırır. MesajVer() fonksiyonu ise ekranda “Butona tıklandı” mesajı yazadırır.

Javascript ile olay kontrolü ise aşağıdaki gibi yapılabilir.

```
<button id="btn">Tıkla</button>
<script>
let buton = document.getElementById("btn");
buton.onclick = function(){
  alert("Butona tıklandı!");
};
</script>
```

Önce adı btn olan ve üzerinde Tıkla yazan bir buton oluşturulur. Script içerisinde DOM yardımıyla ilgili buton elemanı buton değişkenine atanır. Buton değişkeninin onclick olayı bir fonksiyona bağlanır ve fonksiyon ekranda “Butona tıklandı” mesajı yazdırır.

addEvenListener ile olay kontrolü aşağıdaki gibi yapılır.

```
<button id="btn">Tıkla</button>

<script>
let buton = document.getElementById("btn");

buton.addEventListener("click", function(){
```

```
  alert("Butona tıklandı!");
});
</script>
```

Öncelikle adı btn olan bir buton oluşturulur. Butonun üzerinde Tıkla mesajı yer alır. Script bölümünde buton elemanı DOM yardımıyla buton değişkenine atanır. Buton değişkeninin addEvenListener yöntemi butona uygulanan olay eğer click (tıklama) ise ekranda "Butona tıklandı" mesajını alert fonksiyonu yardımıyla yazdırır.

Aşağıda basit bir mail doğrulama örneği görülmektedir.

```
<input id="mail">
<button onclick="kontrol()">Kontrol</button>

<script>
function kontrol(){
  let m = mail.value;
  if(m.includes("@")){
    alert("Geçerli mail");
  }else{
    alert("Geçersiz mail");
  }
}
</script>
```

5.3.2. Mouse olayı ile ilgili örnekler

```
<div id="kutu"
style="width:200px;height:100px;background:lightblue;">
  Bu bölge üzerinde Mouse gezdir.
</div>

<script>
let kutu = document.getElementById("kutu");
```

```
kutu.addEventListener("mouseover", function(){
  kutu.style.background = "orange";
});
```

```
kutu.addEventListener("mouseout", function(){
  kutu.style.background = "lightblue";
});
</script>
```

Kodun başında adı kutu olan bir div tanımlanır. Tanımlanan bu div, 200 piksel yükseklikte ve 200 piksel genişliktedir. Ayrıca kutu isimli bu div alanının geri plan rengi lighblue olarak ayarlanmıştır.

Script bölümünde ise öncelikle az önce tanımlanan kutu DOM yardımıyla kutu değişkenine atanır. Kutu değişkeninin addEvenListener olayında bu sefer “mouseover” (Mouse nesne üzerinde) ve “mouseout” (Mouse nesneyi terk etti) olayları yardımıyla renk değiştirme yapılır.

5.3.3. Klavye ile ilgili olay örnekleri

Keyup olayı ile ilgili örnek aşağıda verilmiştir.

```
<input type="text" id="ad" placeholder="Bir şey yaz">

<script>
let input = document.getElementById("ad");

input.addEventListener("keyup", function(){
  console.log("Yazılan:", input.value);
});
</script>
```

Öncelikle adı “ad” olan bir metin kutusu oluşturulur. Bu metin kutusuna veri girişi yapılırken her seferinde (karakter girişi ya da silme) o anda yazılan kısmı konsolda gösterir.

5.3.4. Hesap makinesi örneği

Aşağıdaki örnek click olayı yardımıyla basit bir hesap makinesi işlevi yerine getirir.

```
<input id="s1">
<input id="s2">
<button onclick="topla()">Topla</button>
<p id="sonuc"></p>

<script>
function topla(){
  let a = Number(s1.value);
  let b = Number(s2.value);
  sonuc.innerText = a + b;
}
</script>
```

Diğer örneklerden farklı olarak document.getElementById("s1").value gibi bir kullanım yerine daha sade bir kullanım tercih edilmiştir.

5.4. LocalStorage kullanımı

```
localStorage.setItem("kullanici","Hidayet");

let veri = localStorage.getItem("kullanici");
console.log(veri);
```

LocalStorage yardımıyla bellekte değişken tutabilir ve değerlerine ulaşabiliriz. Cookie ve Session kullanımlarına alternatif bir yöntem sağlar.

5.5. Fetch API ile veri çekme

Fetch API ile veri çekme, bir arayüz sayesinde **web servislerinden (API)** veri almayı veya sunucuya veri göndermeyi ifade eder. En yaygın kullanım alanı, bir API'den **JSON formatında veri çekmek** ve bunu web sayfasında kullanmaktır.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
.then(res => res.json())
.then(data => console.log(data));
```

Bu konuda sahte veri üreten https://jsonplaceholder.typicode.com/posts adresi örnek olarak kullanılmıştır. Fetch API, HTTP istekleri (GET, POST, PUT, DELETE vb.) yapmayı sağlayan modern bir JavaScript arayüzüdür. Hem gönderim yapan hem de veri alan uygulama aşağıda görüldüğü gibidir:

```
let yeniPost = {
  title: "Fetch Öğreniyorum",
  body: "JSON.stringify ve JSON.parse kullanımı",
  userId: 10
};

// 1) Gönder
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(yeniPost)
})

// 2) Gelen JSON'u JS objeye çevir
.then(res => res.text())
.then(text => {
   let sonuc = JSON.parse(text);
   console.log("Sunucudan Gelen:", sonuc);
});
```

5.6. Hata yakalama

Kodlama yaparken hata meydana gelebilir. Hatanın ne olduğunun tespiti ya da hata nedeniyle program kesintisi yaşamamak için hata yakalama rutinleri çalıştırılır. Javascriptte bu sorun aşağıdaki gibi çözülür.

```
try {
  risky();
} catch (e) {
  console.error(e.message);
}
```

Hata çıkarma ihtimali olan kod try {} bölümüne yazılır. Yukarıdaki örnekte hata ayıklaması yapılacak fonksiyon risky() fonksiyonudur. Fonksiyon içerisinde tanımsız bir değişken kullanılmış olabilir ya da runtime hatası oluşabilir. Bir hata olması durumunda catch (e) bloğuna gider ve o blokta hatanın ne olduğu konsolda gösterilir.

Hata olsa bile çalışan bir uygulama için de aşağıdaki yazılır:

```
try {
  risky();
} catch (e) {
  console.error(e.message);
} finally {
  console.log("Bu her durumda çalışır");
}
```

5.7. Modüle kullanımı

```
// math.js
export const sum = (a, b) => a + b;

// app.js
import { sum } from "./math.js";

// app.js içinden math.js içindeki fonksiyonlara ulaşılır.
```

```
<script type="module" src="app.js"></script>
// app.js dosyası kullanılır.
```

Yukarıdaki kodun;

```
export const sum = (a, b) => a + b;
```

Bölümünde `sum` adında bir **fonksiyon tanımlanıyor ve** `export` anahtar kelimesi ile bu fonksiyon **dosya dışına açılıyor.** `export` olmazsa, dışarıdan **erişilemez**.

app.js dosyasındaki aşağıdaki satır;

```
import { sum } from "./math.js";
```

aynı dizindeki `math.js` dosyasını gösterir. `{ sum }` → `math.js` içinden **ismi birebir aynı olan** `sum` fonksiyonunu alır. Artık `app.js` içinde `sum()` kullanılabilir.